

UNIDAD 1

1. INTRODUCCIÓN A LA INGENIERÍA DEL SOFTWARE. ¿QUÉ ES?

Software: NO ES SOLO CÓDIGO fuente o ejecutables, sino que es un conjunto de programas ejecutables y código fuente + datos y archivos de configuración + documentación del sistema y del usuario + herramientas para la construcción y mantenimiento. Se define formalmente como CONOCIMIENTO REPRESENTADO EN DISTINTOS NIVELES DE ABSTRACCIÓN, evoluciona desde la toma de requerimientos hasta su representación física en código ejecutable.

Ingeniería de software: disciplina tecnológica y de gestión que aplica un enfoque sistemático, disciplinado y cuantificable al análisis, diseño, construcción, validación y mantenimiento en los sistemas informáticos. Se ocupa de todos los aspectos de la producción del software profesional (procesos, métodos y herramientas sobre una cultura de calidad) para transformar conocimiento abstracto en productos confiables, eficientes y mantenibles que satisfagan las necesidades del cliente.

Se concibe como una tecnología estructurada en cuatro capas fundamentales:

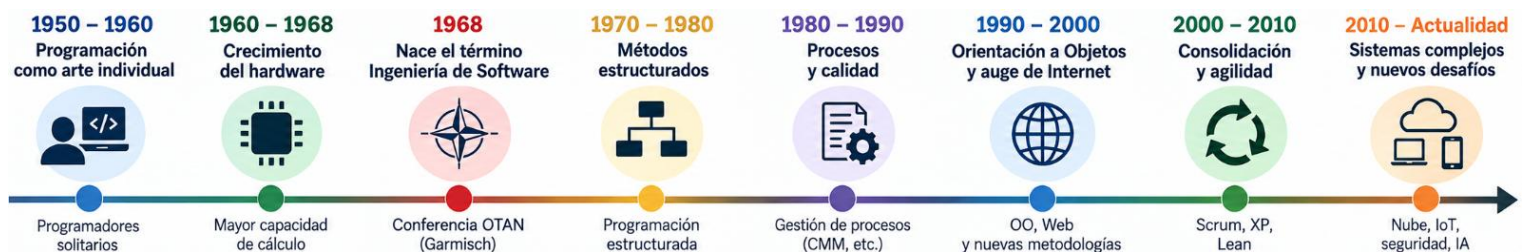
1. Compromiso con la calidad: cultura de mejora continua de la organización
2. Proceso: establece el marco para la administración, hitos aseguramiento de calidad y control de cambios del proyecto.
3. Métodos: proporcionan experiencia técnica para elaborar el producto.
4. Soporte para procesos y métodos.

Premisas fundamentales:

- Saber programar no es suficiente: se requiere comprender el problema del cliente antes de proponer una solución.
- El diseño es crucial: es la actividad clave para garantizar la calidad del producto y su mantenimiento a largo plazo.
- Organización del trabajo: involucra disciplinas técnicas, de gestión y soporte.

2. ESTADO ACTUAL Y ANTECEDENTES. LA CRISIS DEL SOFTWARE

El Origen de la disciplina: en el 50-60 el desarrollo de programas era una actividad artesanal realizado por programadores solitarios. Mientras que el hardware reducía sus costos y mejoraba sus prestaciones, la construcción del software no sufrió un auge similar porque es una actividad humana intensiva cuyo producto es abstracto e intangible. En 1968 nace el término Ingeniería de Software y Crisis del software de la mano de Friedrich Bauer en una conferencia de OTAN (Dijkstra ya había anticipado los términos en el escrito “El humilde programador”). En los 70s-80s surgen métodos estructurados, la programación estructurada y el ocultamiento de información; además Frederick Brooks publica The Mythical Man-Month (75) y No Silver Bullet (89). De los 90s en adelante aparece POAO, Internet, Web, metodologías ágiles, etc.



La Crisis del software: conjunto de dificultades, fallas y problemas vinculados a la planificación, estimación de costos, productividad y calidad en el desarrollo de productos informáticos. Se manifiesta históricamente en un conjunto de síntomas críticos que evidencian la falta de madurez de la disciplina: en primer lugar, tenemos las desviaciones de presupuesto y tiempo (proyectos entregados fuera de plazo, sobrecostos económicos), a esto se le suma baja calidad y presencia de defectos (productos inestables y difíciles de mantener). Como consecuencia directa de esto, se produce una gran insatisfacción por parte del cliente (el SW no responde a las necesidades del usuario y el negocio) y además la mala documentación deriva en un mantenimiento complejo (actualizaciones podrían romper su funcionamiento).

El origen de la crisis tiene lugar en el desfase entre el SW y el HW que permitió crear sistemas cada vez más grandes sin contar con métodos adecuados para desarrollar el software. A esto se sumaron la subestimación de la complejidad, los requerimientos ambiguos o cambiantes, la falta de gestión y control de calidad (SCM) y la creencia de que solo saber programar era suficiente para desarrollar sistemas complejos.

Consecuencias y errores históricos emblemáticos:

- Mariner I (1962): Un error en una instrucción utilizada por el software de navegación provocó que la nave se desviara de su trayectoria. Por seguridad, tuvo que ser destruida poco después del lanzamiento.
- Hartford Coliseum (1978): Errores en el análisis y diseño estructural asistido por computadora contribuyeron a que se subestimaran ciertas cargas. Como consecuencia, el techo del estadio colapsó.
- Ariane 5 (1996): El software intentó convertir un número de 64 bits a uno de 16 bits, pero el valor era demasiado grande. El error afectó el sistema de navegación y el cohete terminó autodestruyéndose.
- Mars Climate Orbiter (1999): Dos equipos utilizaron sistemas de unidades diferentes: uno trabajaba con unidades inglesas y otro con el sistema métrico. Esto produjo cálculos incorrectos de trayectoria y la pérdida de la sonda.

- Error del Milenio (Y2K): Muchos programas almacenaban el año con solo dos dígitos. Así, el año 2000 aparecía como “00” y podía confundirse con 1900, generando riesgo de errores en sistemas bancarios, administrativos y otros sistemas informáticos.

Estado Actual: la IS ha madurado, existen estándares de calidad, marcos de evaluación de procesos, herramientas CASE y enfoques ágiles y lean consolidados. Sin embargo, los problemas con los proyectos no han desaparecido por completo. Los desafíos actuales se deben a: demandas crecientes (hay una mayor exigencia por parte de la sociedad, además de tiempos de entrega e implementación reducidos) y expectativas bajas (ya que la mayoría de las empresas aun desarrollan SW de manera informal sin aplicar métodos profesionales, conformándose con que el código funcione). Los principales desafíos actuales son integrar sistemas antiguos con nuevas tecnologías, adaptar rápidamente el software a los cambios y garantizar que los sistemas sean seguros, confiables y resistentes a fallas.

3. DISCIPLINAS QUE CONFORMAN LA INGENIERÍA DE SOFTWARE

A nivel de estándar profesional, el SWEBOK (*Software Engineering Body of Knowledge*) formalizado por la IEEE organiza el cuerpo de conocimiento de la profesión en 15 áreas de conocimiento (*Knowledge Areas*).

Disciplinas técnicas: actividades orientadas directamente a la creación, evolución y construcción del SW como producto. Encontramos:

- Toma y Análisis de Requerimientos: Relevamiento, especificación y validación de las necesidades del cliente y restricciones del sistema.
- Análisis y Diseño de Software: Modelado técnico y definición de la arquitectura del sistema.
- Construcción / Implementación: Codificación de la solución a partir de los modelos de diseño.
- Pruebas de Software (Testing): Ejecución de ensayos para la detección de defectos y verificación del funcionamiento correcto del producto.
- Despliegue (Deployment): Instalación y puesta en marcha del producto en el entorno de producción.
- Documentación y Capacitación: Elaboración de manuales técnicos y de usuario, junto con el entrenamiento a los usuarios finales.

Disciplinas de Gestión (Project Management): actividades enfocadas en organizar, dirigir, monitorear y controlar el proyecto de desarrollo de software. Permiten adaptar el proceso a las características del proyecto y a las necesidades del equipo. Incluyen:

- Planificación de Proyecto: Estimación de tiempos, costos, esfuerzos y recursos, así como la confección del plan de proyecto.
- Monitoreo y Control de Proyectos: Seguimiento continuo del progreso y gestión de desviaciones mediante hitos o iteraciones.

Disciplinas de Soporte (Transversales, Protectoras o "Sombrilla"): disciplinas independientes del marco conceptual que recubren al proceso de desarrollo y se aplican de manera transversal a lo largo de todo el ciclo de vida para verificar y garantizar la integridad y calidad del producto y del proceso. Incluyen:

- Gestión de Configuración de Software (SCM): Identificación, trazabilidad y control de cambios en todos los ítems de configuración a lo largo del ciclo de vida.
- Aseguramiento de la Calidad de Software (QA / SQA / PPA): Definición y auditoría de procesos para garantizar que se cumplan los estándares de calidad.
- Toma de Métricas: Medición cuantitativa del proceso, del proyecto y del producto para fundamentar la toma de decisiones.

4.EJEMPLOS DE GRANDES PROYECTOS DE SOFTWARE FALLIDOS Y EXITOSOS

Grandes proyectos fallidos: Entre los casos más conocidos está Ariane 5 (1996), que explotó por un error de conversión numérica en software reutilizado; Mars Climate Orbiter (1999), perdido por utilizar diferentes sistemas de unidades; Mariner I (1962), destruido por un error en una fórmula; Hartford Coliseum (1978), cuyo techo colapsó por errores de cálculo en el diseño; Toyota (2010), que retiró millones de vehículos por problemas vinculados al software; y Virtual Case File del FBI (2005), cancelado después de grandes inversiones debido a problemas de gestión y cambios constantes.

Grandes proyectos exitosos: Entre los casos exitosos se encuentra la Línea 14 del Metro de París, automatizada y con software altamente verificado; los sistemas de vuelo de Boeing 777 y Airbus A340, diseñados con redundancia y tolerancia a fallas; el software del Space Shuttle de la NASA, reconocido por sus estrictos procesos de prueba y control de calidad; y proyectos de código abierto como Linux, Apache y MySQL, que alcanzaron gran estabilidad mediante el desarrollo y revisión colaborativa.

5.CICLOS DE VIDA (MODELOS DE PROCESO) Y SU INFLUENCIA EN LA ADMINISTRACIÓN DE PROYECTOS DE SOFTWARE

Ciclo de vida: representación simplificada de un proceso que grafica la serie de pasos por las que un producto o proyecto progresa en el tiempo. Especifica las fases de desarrollo y el orden secuencial y/o iterativo en el que se lleva a cabo. Determina como y cuando se ejecutan las actividades dentro de un proyecto, estableciendo esfuerzo y puntos de control. Un mismo proceso puede estar en distintos ciclos de vida.

Ciclo de vida del Proyecto: ventana temporal de desarrollo (desde la etapa inicial hasta la entrega final del producto).

Ciclo de vida del Producto: Abarca desde la concepción de la idea y el plan de negocio, pasa por la etapa de desarrollo, y continúa a lo largo de su operación, mantenimiento, actualizaciones y parches hasta su retiro definitivo del mercado. Es mas amplio que el del proyecto. Un mismo producto puede incluir varios ciclos de vida de proyectos (actualizaciones/evoluciones).

Tipos de ciclos de vida:

- Secuencial/Lineal-Cascada: orden lineal estricto. Exige completar el 100% de una fase y realizar una revisión o hito de aprobación para avanzar. *Modelo Concreto: Modelo en Cascada (Waterfall)*
- Iterativo e Incremental: divide el producto en incrementos. En cada iteración se ejecutan actividades de requerimientos, análisis, diseño, construcción y pruebas para obtener un incremento de software funcionando. Se utiliza en metodologías ágiles. *Modelos Concretos: Proceso Unificado (PUD / RUP)* (ciclo iterativo dentro de un marco definido) y marcos ágiles como Scrum o XP
- Recursivo/Evolutivo: organiza el proyecto en bucles o ciclos dirigidos por la evaluación de riesgos. Cada circuito representa una base evolutiva que mitiga incertidumbres antes de planificar la siguiente iteración. *Modelo Concreto: Modelo en Espiral de Boehm*

La elección del modelo de ciclo de vida condiciona la gestión del proyecto en cinco aspectos clave:

- Planificación y Control: En modelos secuenciales, la planificación es inicial y masiva (*up-front*), controlando el avance mediante documentos. En iterativos, la planificación es continua y adaptativa, midiendo el progreso con software funcionando.
- Triple Restricción (Alcance, Tiempo, Costo): Los enfoques tradicionales definen el alcance al inicio (cualquier cambio altera costos y plazos), mientras que los ágiles fijan tiempo y recursos (*time-boxing*) manteniendo el alcance flexible y re-priorizable.
- Gestión de Riesgos: Frente a alta volatilidad o incertidumbre técnica, los ciclos iterativos reducen el impacto al permitir fallar temprano y corregir el rumbo rápidamente.
- Involucramiento del Cliente y Entregas: Determina la frecuencia de retroalimentación; los esquemas en cascada entregan el valor al final del proyecto, mientras que los iterativos aportan valor e incrementos de forma constante.
- Compatibilidad Metodológica: Los procesos empíricos (como Scrum) requieren obligatoriamente ciclos iterativos-incrementales para habilitar la inspección y adaptación; forzarlos en esquemas secuenciales genera una incompatibilidad de gestión.

6.PROCESOS DE DESARROLLO EMPÍRICOS VS. DEFINIDOS

Procesos Definidos (Predictivos/Deterministas): se inspiran en procesos industriales y parten de la idea de que, siguiendo de manera repetida y estandarizada los mismos pasos, se pueden obtener resultados predecibles. Son más adecuados cuando los requerimientos son conocidos y relativamente estables, y el control se basa principalmente en el cumplimiento del plan. Tienen la flexibilidad de admitir cualquier tipo de ciclo de vida: secuencial (cascada), iterativo e incremental (como el PUD) o recursivo (espiral).

Procesos Empíricos (Adaptativos): reconocen que el desarrollo de software es una actividad compleja, creativa y cambiante, donde no es posible predecir completamente los resultados. Por eso, el control se basa en la transparencia, inspección y adaptación, permitiendo ajustar continuamente el producto frente a cambios en los requerimientos. Por su propia naturaleza, exigen obligatoriamente un ciclo de vida iterativo e incremental. Se prohíbe el uso de un ciclo puramente secuencial en el enfoque empírico, ya que diferir la retroalimentación y la validación hasta el final impide el aprendizaje y la adaptación continuos.

Tipo de Proceso	Procesos Definidos	Procesos Empíricos
Mecanismo de control	La administración y el control provienen de la predictibilidad del proceso y del cumplimiento de un plan previamente establecido	La administración y el control no provienen de un plan rígido, sino del control empírico de procesos basado en la inspección frecuente y la adaptación rápida. Se apoyan en los pilares de transparencia, inspección y adaptación
Premisa de calidad	Asumen que el proceso es el único factor totalmente controlable; por lo tanto, si se diseña un proceso de alta calidad y se sigue rigurosamente, el producto resultante será de calidad	La calidad no proviene de seguir ciegamente un proceso escrito, sino de obtener software funcionando en ciclos cortos, inspeccionarlo e introducir mejoras continuas (pensamiento Lean para eliminar actividades que no aportan valor)
Uso de experiencia	Consideran que la experiencia e información obtenidas en proyectos pasados se pueden sistematizar y extrapolar a futuros proyectos o diferentes equipos para estimar tiempos y costos de manera predecible	Asumen que la experiencia no es directamente extrapolable entre proyectos o equipos distintos. El conocimiento se genera empíricamente a través de la vivencia directa del equipo dentro de su propio contexto
Gestión del cambio	Los cambios son analizados e incorporados dentro de una estructura y planificación prefijadas, evaluando previamente su impacto sobre el plan	Adoptan una postura de alta adaptabilidad; buscan detectar y responder rápidamente a los cambios mediante iteraciones cortas e incrementos de producto con retroalimentación frecuente

7.CICLOS DE VIDA (MODELOS DE PROCESO) Y PROCESOS DE DESARROLLO DE SOFTWARE

Ciclo de Vida (Modelo de proceso): representación abstracta o simplificada del proceso que grafica los pasos a través de los cuales el producto o proyecto avanza en el tiempo. Especifica las fases y orden del proceso. Define el CÓMO y CUÁNDO se ejecutan las actividades de un proyecto específico.

Proceso de desarrollo de software: conjunto de actividades, métodos, prácticas, herramientas y transformaciones que utiliza el equipo para desarrollar y mantener el software y sus productos asociados. Define el QUÉ se debe hacer, indicando entradas, salidas, tareas, herramientas y roles necesarios.

8.VENTAJAS Y DESVENTAJAS DE C/U DE LOS CICLOS DE VIDA

CICLO DE VIDA SECUENCIAL	
VENTAJAS	DESVENTAJAS
Estructura clara y predecible. Facilita la planificación y el control. Útil con requerimientos estables. Permite definir detalladamente el sistema antes de programar.	Poco adaptable a cambios. Software funcional recién al final. Errores tardíos generan mucho retrabajo. Puede generar tiempos de espera entre etapas.

CICLO DE VIDA ITERATIVO E INCREMENTAL	
VENTAJAS	DESVENTAJAS
Se adapta fácilmente a cambios. Retroalimentación continua del cliente Entrega valor desde etapas tempranas. Detecta errores y riesgos antes.	Menor previsibilidad inicial. Requiere refactorización para mantener la arquitectura. Puede dificultar contratos de alcance fijo. Requiere participación frecuente del cliente/equipo.

CICLO DE VIDA RECURSIVO	
VENTAJAS	DESVENTAJAS
Excelente gestión de riesgos. Adecuado para sistemas grandes y complejos. Permite utilizar prototipos. Se adapta a la evolución del producto.	Mayor costo y complejidad. Requiere expertos en análisis de riesgos. Difícil estimar costo y duración total. Puede ser excesivo para proyectos pequeños.

MODELOS INFORMALES	
VENTAJAS	DESVENTAJAS
Desarrollo inicial rápido. Poca planificación y documentación. La prototipación ayuda a descubrir requerimientos. Útiles para prototipos y proyectos pequeños.	Difíciles de planificar y mantener. Riesgo de mala arquitectura y baja calidad. Difícil estimar tiempos de entrega. El prototipo puede terminar utilizándose como producto final.

Incluyen enfoques con poca formalización, como Code and Fix, y modelos específicos como la Prototipación Evolutiva.

9.CRITERIOS PARA ELECCIÓN DE CICLOS DE VIDA EN FUNCIÓN DE LAS NECESIDADES DEL PROYECTO Y LAS CARACTERÍSTICAS DEL PRODUCTO

La selección del modelo de ciclo de vida es una decisión estratégica que depende directamente de la evaluación de los niveles de riesgo, la naturaleza del producto y el contexto del proyecto. No existe un ciclo de vida “correcto” por lo que se deben ponderar un conjunto de factores de decisión para determinar la opción mas conveniente.

Se deben tomar en cuenta las siguientes características clave:

- Claridad y estabilidad de los requerimientos: evalúa si las necesidades del negocio están bien definidas o si presentan alta volatilidad e incertidumbre.
- Nivel de riesgo e incertidumbre: mide grado de complejidad de la técnica, la novedad tecnológica y el impacto financiero o critico en caso de fallas.
- Urgencia del cliente y tiempo de entrega (Time-to-market): determina si el cliente necesita disponer de funcionalidades básicas en el corto plazo o si puede esperar a la entrega final completa.
- Factibilidad de entregas parciales: analiza si la arquitectura del producto permite construir y desplegar incrementos de forma periódica.
- Disponibilidad e involucramiento del cliente: mide la capacidad del cliente para participar en revisiones continuas, validar entregables parciales y aportar retroalimentación.
- Competencias de equipo y cultura organizacional: considera los conocimientos del equipo y si la organización opera en esquemas tradicionales o ágiles.

Criterios de elección según el ciclo de vida

Ciclo de vida	Características del producto	Necesidades del proyecto	Entregas	Cliente	Prioridad
Secuencial	Requerimientos claros, completos y estables. Baja incertidumbre. Producto difícil de dividir.	Planificación previa, documentación formal y pocos cambios.	Entrega única al final.	Al inicio y cierre.	Cumplir el plan.
Iterativo e Incremental	Requerimientos incompletos o cambiantes. Producto divisible en incrementos.	Resultados tempranos, adaptación y validación rápida.	Entregas frecuentes.	Feedback continuo.	Adaptarse y entregar valor.

Recursivo / Evolutivo	Sistemas grandes, complejos o críticos, con incertidumbre técnica.	Analizar alternativas y reducir riesgos mediante prototipos y simulaciones.	Prototipos evolutivos por ciclo.	Revisión en cada ciclo.	Reducir riesgos.
------------------------------	--	---	----------------------------------	-------------------------	------------------

10.COMPONENTES DE UN PROYECTO DE SISTEMAS DE INFORMACIÓN.

Proyecto: es un esfuerzo temporal y organizado que combina RRHH, técnicos y metodológicos para construir un producto, servicio o resultado único que aporte valor al negocio. Es la unidad organizativa mediante la cual se adapta e instancia el marco teórico del proceso a un contexto específico de desarrollo, integrando a las personas y definiendo los mecanismos para evaluar la calidad, gestionar riesgos y controlar cambios. Un proyecto se caracteriza por ser:

- Único e irreplicable: el resultado del proyecto es singular. Dos proyectos similares difieren en calendarios, presupuestos, equipos, entornos, etc.
- OAO: se dirige a obtener resultados concretos mediante objetivos claros, no ambiguos, medibles y alcanzables.
- Duración limitada en el tiempo: debe tener principio y fin definidos. Al alcanzar los objetivos, el proyecto concluye y los recursos se liberan, lo que lo diferencia de las operaciones continuas o líneas de producción.
- Conjunto de tareas interrelacionadas basadas en esfuerzo y recursos: se definen actividades con dependencias mutuas, asignando personas y equipamiento para dominar la complejidad inherente al desarrollo.

Administración del proyecto: es la aplicación de conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para organizar y coordinar recursos humanos y materiales. Su propósito es lograr el desarrollo del producto dentro del tiempo, costo y requerimientos acordados. Abarca dos grandes dimensiones de gestión: Planificación (definir qué, cómo, cuando y quien) y Monitoreo y control (medir avances y desviaciones).

Atributos particulares de la administración de proyectos de software:

- El producto es intangible: resulta complejo medir el progreso real sin artefactos intermedios.
- Espectro excepcional y novedoso: los proyectos suelen ser únicos, la experiencia previa no siempre es extrapolable debido al rápido cambio tecnológico.
- Procesos específicos de la organización: resulta difícil predecir con certeza matemática que proceso estandarizado funcionará mejor sin adaptarlo al equipo.

La Triple Restricción: la gestión tradicional sostiene que un proyecto se equilibra sobre tres variables que condicionan la calidad:

- Alcance: Qué funcionalidades e ítems se van a construir.
- Tiempo: La fecha de entrega o ventana temporal del calendario.
- Costo / Recursos: Presupuesto, equipamiento, infraestructura y salarios del personal.



Comportamiento del triángulo: La Calidad no se negocia. Si se modifica una variable de la triple restricción sin ajustar las otras (por ejemplo, aumentar el alcance sin dar más tiempo ni presupuesto), el impacto negativo recae directamente sobre la calidad del producto:

Diferencia de enfoque (Tradicional vs. Ágil):

- Enfoque Tradicional: Se fija el Alcance al inicio (mediante los requerimientos); a partir de allí se estiman el Tiempo y el Costo.
- Enfoque Ágil: Se fijan el Tiempo (sprints/timeboxes) y el Costo/Recursos (equipo de trabajo fijo), mientras que el Alcance permanece variable y adaptable a la retroalimentación.

Equipo de proyecto: grupo de personas con habilidades y especialidades diversas (desarrolladores, analistas, arquitectos, testers, responsables de calidad y configuración) comprometidos con alcanzar objetivos por los cuales se sienten mutuamente responsables. Características de un equipo cohesivo: Comunicación fluida, estabilidad en su composición, desarrollo de estándares propios de calidad, aprendizaje mutuo y sentido de responsabilidad compartida como una sola unidad.

Líder de proyecto: Es el responsable de la toma de decisiones, la elaboración del plan, la asignación de tareas y el balanceo continuo de las variables de la triple restricción. Deben caracterizarse por tener aptitudes y cualidades como Motivación, organización, capacidad de innovación, empatía, comunicación efectiva, liderazgo, resolución de problemas técnicos e identidad administrativa. Sus responsabilidades son: Definir el alcance, identificar a los involucrados, estructurar las tareas, realizar estimaciones, gestionar los riesgos y administrar los mecanismos de control de cambios.

Stakeholders: interesados o involucrados cuyos intereses pueden verse afectados por el proyecto o cuyas acciones influyen en su resultado. Incluye al cliente, usuarios finales, patrocinador (*sponsor*), equipo de dirección, líder de proyecto, equipo de desarrollo, gerentes funcionales, subcontratistas y organismos regulatorios.

Gestión de expectativas: Una función clave de la administración es adaptar los planes y el enfoque para conciliar los diferentes intereses y necesidades de estos actores.

Plan de Proyecto: es el artefacto formal de gestión que funciona como la "hoja de ruta" del desarrollo. Preguntas clave que responde:

- ¿Qué se hace? (Actividades para cubrir el alcance)
- ¿Cuándo se hace? (Calendario y cronograma)
- ¿Cómo se hace? (Procesos, métodos, herramientas y planes de soporte)
- ¿Quién lo hace? (Asignación de roles y responsabilidades)

Secuencia lógica de planificación:

Objetivos y Alcance → Proceso y Ciclo de Vida → Estimación de Tamaño → Estimación de Esfuerzo → Calendario → Costo → Cronograma

El cronograma definitivo se obtiene hacia el final de la secuencia, solo cuando se conoce con claridad qué trabajo debe realizarse, cuánto esfuerzo representa y con qué recursos se cuenta.

Responder a estas preguntas, implica las siguientes actividades:

a. Definición del Alcance (Producto vs. Proyecto): El alcance responde a qué se está desarrollando y constituye la frontera conceptual y operativa del trabajo. Es fundamental distinguir sus dos dimensiones:

- Alcance del Producto: Comprende todas las características, funcionalidades, datos, comportamientos y métricas de desempeño que deben estar presentes en el software a entregar al cliente. Su cumplimiento se mide estrictamente contra la Especificación de Requerimientos de Software (ERS).
- Alcance del Proyecto: Representa todo el trabajo y solo el trabajo necesario para construir y entregar el producto con las características especificadas.

Incluye las actividades técnicas, de gestión y de soporte, como: Relevamiento, Diseño, Codificación, Testing, Despliegue, Gestión de Configuración de Software (SCM) y Otras actividades necesarias para completar el proyecto.

Su cumplimiento se mide contra el Plan de Proyecto.

Relación entre ambos: El alcance del producto condiciona el alcance del proyecto. Sin embargo, una modificación en el alcance del proyecto —por ejemplo, reducir el proyecto a entregar únicamente un documento de especificación de requerimientos o un prototipo— no altera la definición conceptual del producto.

b. Definición de Proceso y Ciclo de Vida: Esta actividad establece el marco metodológico y la estructuración temporal de la ejecución.

Proceso de Software: Es la secuencia estructurada de actividades, métodos, herramientas y roles que definen el “qué se hace” de forma abstracta. Se instancia y adapta según la complejidad y el contexto particular del proyecto.

Ciclo de Vida (Modelo de Proceso): Es el marco organizativo que determina el “cómo y cuándo” se ejecutan dichas actividades en el tiempo, definiendo:

- La secuencia de fases.
- La cadencia de entregas.
- La distribución del esfuerzo.
- Los hitos de control.

Compatibilidad: Los enfoques tradicionales/definidos admiten ciclos de vida: Secuenciales (cascada) - Iterativos/incrementales - Recursivos (espiral). Los enfoques empíricos/ágiles requieren obligatoriamente un ciclo de vida iterativo e incremental para habilitar la inspección y adaptación continua.

c. Estimación de Software: La estimación es el proceso predictivo que busca prever la cantidad de trabajo, tiempo y recursos requeridos antes de la ejecución. Se fundamenta en una cadena secuencial de dependencia obligatoria: Tamaño → Esfuerzo → Calendario → Costos

Tamaño: Estimación de la magnitud del software a construir, medida en unidades como:

- Líneas de Código (LOC).
- Puntos de Función (PF).
- Puntos de Historia (SP).

Esfuerzo: Cantidad total de trabajo intelectual necesario para desarrollar ese tamaño, expresado en horas-persona lineales (dedicación neta sin interrupciones). Es el factor determinante del costo —representa cerca del 80 %— y depende fuertemente de la experiencia y habilidades del equipo.

Calendario (Duración): Tiempo real de duración del proyecto en meses o semanas, derivado de distribuir el esfuerzo según: El tamaño del equipo. la disponibilidad real y las tareas paralelas.

Costos: Valor monetario final que resulta de multiplicar las horas de esfuerzo por la tarifa por hora, según roles o perfiles, sumado a los costos de: Infraestructura+ Herramientas + Recursos críticos.

d. Gestión de Riesgos: Es el proceso sistemático y proactivo enfocado en manejar la incertidumbre para identificar, evaluar y controlar eventos potenciales que podrían comprometer el éxito del proyecto. Esto se diferencia de un problema, que es una certeza o un evento que ya ocurrió.

Exposición al Riesgo (ER): Es una medida cuantitativa de la importancia de un riesgo, calculada como:

Exposición al Riesgo (ER) = Probabilidad de Ocurrencia (P) × Impacto o Severidad (I)

Permite comparar y priorizar en qué riesgos debe focalizar sus recursos la administración.

Etapas de la Gestión Proactiva

- 1) Identificación: descubrir amenazas potenciales al producto, proceso u organización.
- 2) Análisis: valorar la probabilidad y el impacto para determinar la exposición.
- 3) Planificación de Respuestas: Plan de Mitigación: acciones preventivas previas para reducir la probabilidad o el impacto de ocurrencia. Plan de Contingencia: plan de acción diseñado para ejecutarse si el riesgo finalmente se materializa, permitiendo reaccionar de forma controlada y volver a la normalidad.
- 4) Monitoreo y Control: seguimiento continuo durante todo el ciclo de vida para verificar si las suposiciones o los niveles de riesgo han variado.

e. Asignación de Recursos / Responsabilidades: consiste en la distribución planificada del equipo de trabajo y de la infraestructura necesaria sobre las distintas tareas del proyecto. Más que hablar de “recursos humanos” intercambiables, se habla de asignación de personas y responsabilidades.

Manejo de Roles: Asigna los roles del proceso a los integrantes del equipo según sus perfiles y disponibilidad. Por ejemplo: Analista. Desarrollador. Tester. Gestor SCM. En proyectos medianos o pequeños, una misma persona puede asumir múltiples roles.

Restricción temporal: Se debe garantizar que no exista una sobreasignación de personas dentro de una misma ventana de tiempo.

f. Programación de Proyectos (Scheduling / Calendarización): Consiste en transformar la planificación conceptual y las estimaciones en un cronograma operativo detallado.

Mecanismo: Se establecen Fechas específicas de inicio y fin para cada tarea, Interdependencias entre actividades, determinando qué tarea debe

Artefactos habituales: la programación puede formalizarse mediante Diagrama de Gantt, Redes de tareas (PERT/CPM) y en marcos ágiles, planificación de Sprints y tableros como Sprint Backlog o Kanban.

g. Definición de Métricas: Consiste en definir el conjunto de medidas numéricas objetivas que se recolectarán durante el desarrollo para proporcionar visibilidad sobre el estado, avance y calidad del trabajo. Clasificación de Métricas:

- Métricas de Producto: enfocadas en lo que se construye. Ejemplos: Tamaño en LOC/PF/SP. Cobertura de pruebas. Densidad de defectos.
- Métricas de Proyecto: enfocadas en la gestión. Ejemplos: Recursos, Esfuerzo. Desviaciones de calendario, Velocidad, Costos.
- Métricas de Proceso: enfocadas en medir la efectividad global del proceso de software en la organización a largo plazo. Ejemplos: Productividad y Eficiencia de remoción de defectos.

h. Definición de Controles: Consiste en planificar e implementar los mecanismos de seguimiento y verificación que aseguren que la ejecución real del proyecto se mantenga en concordancia con lo planificado, utilizando las métricas como insumo para la toma de decisiones. Mecanismos de Control:

- Revisiones y Reportes: definición de reuniones periódicas de seguimiento —daily, quincenales, etc.—, minutas de reunión y reportes de estado o avance.
- Análisis de Desviaciones: utilización de técnicas cuantitativas como: Análisis de Valor Ganado (AVG) y Gráficos de avance (*Burndown/Burnup Charts*).
- Alineación con Planes de Soporte: controles transversales derivados de: Plan de Gestión de Configuración: control de cambios y líneas base. Plan de Aseguramiento de Calidad (PPQA). Plan de Pruebas.

11.VINCULO PROCESO-PROYECTO-PRODUCTO EN LA GESTIÓN DE UN PROYECTO DE DESARROLLO DE SOFTWARE.

El **vínculo proceso-proyecto-producto** es la relación conceptual y operacional que explica cómo la Ingeniería del Software transforma requerimientos e ideas abstractas en productos de software funcionales. En la gestión del desarrollo de software, estos tres elementos no actúan de forma aislada, sino que se articulan dinámicamente junto con las personas y las herramientas .



- Proceso: marco conceptual abstracto que define cómo se construye el software. Establece el conjunto estructurado de actividades, métodos, prácticas y transformaciones necesarias para desarrollar y mantener el sistema.
- Producto: es el resultado final que se entrega al cliente para aportar valor al cliente. No es solo código ejecutable, abarca todos los artefactos e ítems de configuración asociados (requerimientos, arquitectura, manuales, BDs y casos de prueba).

- Proyecto: es la unidad de gestión y de administración del trabajo. Enmarca el esfuerzo temporal en el que se organizan y coordinan las personas y los recursos para alcanzar los objetivos en un tiempo y presupuesto determinado.
- Personas: ocupa el primer lugar indispensable porque el desarrollo de software es una **actividad intensamente humana e intelectual. Abarca a todos los involucrados (*stakeholders*: gerentes, líderes de equipo, desarrolladores, clientes y usuarios finales) organizados en estructuras de trabajo (tradicionales o ágiles autogestionadas) cuyas habilidades, motivación, cohesión y comunicación determinan directamente la eficiencia del proceso y la calidad del producto final.

"EL PROCESO (AUTOMATIZADO CON HERRAMIENTAS) SE ADAPTA AL PROYECTO, AL CUAL SE INCORPORAN LAS PERSONAS; Y DE ESTA ARTICULACIÓN SE OBTIENE COMO RESULTADO EL PRODUCTO"

El Proceso de ADAPTA al Proyecto: no existe un proceso rígido o universal. El proceso estándar de la organización se instancia y se ajusta según el contexto del proyecto, la complejidad del producto, los objetivos de negocio y la modalidad de trabajo en equipo.

El Proyecto INCORPORA a las Personas: las personas son el corazón del proyecto. Dado que el software es una actividad humano-intensiva, el proyecto actúa como medio para gestionar el talento, la motivación y colaboración del equipo técnico y de los stakeholders.

El equipo UTILIZA las Herramientas para ejecutar el Proceso: las personas aplican las actividades del proceso adaptado utilizando herramientas y equipos tecnológicos que automatizan tareas.

La ejecución del proyecto da como resultado directo el Producto de software con el nivel de calidad esperado.

12.NO SILVER BULLET (FREDERICK BROOKS – 1987)

El ensayo No Silver Bullet: Essence and Accidents of Software Engineering analiza la verdadera naturaleza del desarrollo de sistemas y explica por qué ninguna tecnología o técnica de gestión puede resolver, por sí sola, todos los problemas de esta disciplina ni producir un aumento extraordinario de la productividad. Para explicar esta idea, Brooks utiliza la metáfora del hombre lobo. Así como una persona aparentemente normal puede transformarse de repente en una criatura aterradora, un proyecto de software que al comienzo parece sencillo puede convertirse en una pesadilla caracterizada por retrasos, sobrecostos, errores y fallas de funcionamiento. Ante esta situación, la industria suele buscar una “bala de plata”, es decir, una herramienta o innovación capaz de eliminar rápidamente todos esos problemas. Sin embargo, la tesis central del autor sostiene que no existe ningún avance individual, ya sea tecnológico o administrativo, que pueda garantizar por sí solo una mejora de diez veces en la productividad, la confiabilidad o la simplicidad del desarrollo de software dentro de una década.

Para justificar esta afirmación, Brooks distingue entre dificultades accidentales y dificultades esenciales. Las dificultades accidentales son aquellas relacionadas con la manera en que se produce e implementa el software, pero que no forman parte de su naturaleza más profunda. Entre ellas se encuentran la complejidad de la sintaxis de algunos lenguajes, la programación en lenguaje máquina, la escasez de memoria, las limitaciones del hardware y los largos tiempos de compilación. Gran parte de la evolución histórica de la Ingeniería del Software consistió precisamente en reducir estas dificultades mediante los lenguajes de alto nivel en la década de 1960, los sistemas de tiempo compartido en la década de 1970 y los entornos integrados de desarrollo durante los años ochenta.

Las dificultades esenciales, en cambio, están relacionadas con la construcción mental de la estructura conceptual del software. Esto incluye comprender y especificar los requerimientos, diseñar los algoritmos, organizar las estructuras de datos y establecer las relaciones correspondientes a la lógica del negocio. Desde esta perspectiva, escribir el código no constituye el problema principal. La verdadera dificultad se encuentra en comprender qué debe hacer el sistema y en diseñar y validar una solución que represente correctamente esa necesidad.

Según Brooks, esta esencia del software presenta cuatro características que dificultan su automatización completa: complejidad, conformidad, mutabilidad e invisibilidad.

- Complejidad: el software posee una enorme cantidad de elementos diferentes que interactúan entre sí. Sus partes no suelen repetirse de manera idéntica, ya que, si lo hicieran, podrían reemplazarse mediante una subrutina o un componente reutilizable. Además, a medida que el sistema aumenta de tamaño, su complejidad no crece de manera lineal, sino que las relaciones entre sus elementos se multiplican.
- Conformidad: el software debe adaptarse a numerosas condiciones externas. A diferencia de otras ramas de la ingeniería, no se encuentra gobernado únicamente por leyes universales, como la gravedad o las ecuaciones físicas. Por el contrario, debe ajustarse a requerimientos arbitrarios, regulaciones, interfaces existentes, decisiones empresariales y necesidades humanas que pueden resultar contradictorias o cambiar con el tiempo.
- Mutabilidad: El software se encuentra sometido a una presión constante para ser modificado porque representa una construcción formada por ideas, lo que Brooks denomina “materia de pensamiento puro” (*pure thought-stuff*). Mientras que un producto físico antiguo suele ser reemplazado por uno nuevo, el software acostumbra a modificarse continuamente para acompañar la evolución del negocio, los cambios en el hardware o la aparición de nuevas necesidades.
- Invisibilidad: el software es invisible. No posee una forma física o geométrica que permita observar directamente toda su estructura. Los diagramas de flujo, grafos y otros recursos visuales pueden representar ciertos aspectos del sistema, pero ninguno logra mostrar de manera completa todas las abstracciones y relaciones que lo componen.

En 1987, Brooks también analizó distintas tecnologías que se presentaban como posibles balas de plata, entre ellas la programación orientada a objetos, el lenguaje Ada, las herramientas CASE y la Inteligencia Artificial basada en sistemas expertos. Su conclusión fue que ninguna de ellas conseguiría por sí sola una mejora de un orden de magnitud. En particular, señaló que los sistemas expertos necesitaban que un especialista humano proporcionara previamente el conocimiento y las reglas necesarias. Para Brooks, el desafío fundamental no era expresar una solución mediante código, sino decidir con precisión qué se quería expresar.

En la actualidad, la aparición de la Inteligencia Artificial Generativa volvió a despertar la expectativa de haber encontrado la bala de plata definitiva. No obstante, esta expectativa reproduce la misma ilusión tecnológica de épocas anteriores. La IA puede generar código, completar instrucciones, refactorizar programas y crear pruebas de manera rápida. Estas capacidades mejoran considerablemente la implementación del software, pero actúan principalmente sobre las dificultades accidentales, es decir, sobre el “cómo” se construye una solución.

El cuello de botella continúa siendo la definición del “qué”. La Inteligencia Artificial no comprende automáticamente el problema del cliente, no determina por sí sola cuáles son los requerimientos correctos y tampoco garantiza que el sistema generado resuelva la necesidad adecuada. Incluso cuando la IA puede producir grandes cantidades de código en poco tiempo, depende de que una persona describa correctamente el comportamiento esperado, las restricciones y los criterios de aceptación.

Esta situación produce lo que puede denominarse el “embudo de la especificación”. Mientras que la capacidad de la IA para generar código parece prácticamente ilimitada, la capacidad humana para comprender el problema y transmitirlo de forma precisa continúa siendo el límite del proceso. En el desarrollo guiado por especificaciones (*Spec-Driven Development*) o mediante *prompts*, la persona debe formalizar detalladamente lo que necesita. Si no comprende la complejidad esencial del problema, la especificación será ambigua o incompleta. En consecuencia, la IA podrá producir una respuesta técnicamente correcta, pero basada en requisitos equivocados o insuficientes, ya que no puede deducir con certeza las intenciones que nunca fueron expresadas.

Por esta razón, Brooks no propone esperar una solución milagrosa, sino aplicar mejoras progresivas que permitan enfrentar directamente la esencia del desarrollo de software. Una de ellas es el prototipado rápido, que facilita el descubrimiento gradual de los requerimientos mediante la interacción con el cliente. Esta estrategia reconoce que generalmente es imposible definir una especificación perfecta desde el comienzo y que las necesidades se comprenden mejor al observar y probar versiones preliminares del sistema.

También resulta importante reutilizar o adquirir soluciones existentes, como componentes, *frameworks* o servicios SaaS, en lugar de construir nuevamente aquello que ya se encuentra disponible. Del mismo modo, Brooks destaca la necesidad de formar y cultivar grandes diseñadores, ya que el diseño conceptual es una actividad intelectual cuya calidad depende en gran medida de la experiencia, el criterio y la capacidad de las personas involucradas.

Los enfoques iterativos y ágiles continúan esta misma línea de pensamiento. Estos métodos aceptan que el software es modificable y que los requerimientos pueden evolucionar. Por eso, proponen ciclos cortos de trabajo que permitan inspeccionar los resultados, obtener retroalimentación y adaptar progresivamente el producto.

En conclusión, la propuesta de Brooks sigue siendo vigente incluso frente a los avances actuales de la Inteligencia Artificial. Las nuevas tecnologías pueden reducir considerablemente el esfuerzo necesario para programar, probar y mantener un sistema, pero no eliminan la necesidad de comprender el problema, definir los requerimientos y diseñar una solución apropiada. La Ingeniería del Software no avanza mediante una única “bala de plata”, sino a través de múltiples mejoras complementarias, acompañadas por el conocimiento, el criterio y la capacidad de adaptación de las personas.

UNIDAD 2

13.GESTIÓN DE PRODUCTO

El paradigma de Gestión Lean-Agile de Productos de Software se fundamenta en un cambio de mentalidad: pasar de la administración predictiva centrada en planes y documentación a una gestión adaptativa orientada al valor del cliente.

Filosofía Ágil/Agilismo: no es un proceso, ni un framework, ni una metodología paso a paso. Es una ideología y un pensamiento respaldado por un conjunto de valores y principios que guían la toma de decisiones y el desarrollo de productos. Surgió como respuesta crítica a la rigidez de los procesos tradicionales prescriptivos y en cascada y busca un equilibrio entre el control excesivo y el desarrollo desordenado. Los métodos basados en la filosofía ágil son adaptables y centrados en las personas y reconocen que en los proyectos de SW los requerimientos son volátiles y cambiantes.

El manifiesto Ágil plantea cuatro valores/declaraciones clave, donde AÚN reconociendo los elementos de la derecha, se PRIORIZAN los elementos de la izquierda:

1. Individuos e interacciones por sobre procesos y herramientas: al ser una actividad intensamente humana, en el desarrollo importa más contar con un equipo motivado y comunicación fluida que tener herramientas más sofisticadas.
2. Software funcionando por sobre documentación exhaustiva: la verdadera medida del progreso y entrega de valor al cliente se da con el código operativo y probado, no con la acumulación de documentos.
3. Colaboración con el cliente por sobre negociación contractual: se integra al cliente (o al Product Owner) de forma continua al equipo para ajustar el producto a las necesidades reales, superando la rigidez de los contratos fijos.
4. Responder al cambio por sobre seguir un plan: acepta la volatilidad del contexto como una oportunidad para entregar un producto mas competitivo.

Los valores se operacionalizan a través de 12 principios de agilidad:

1. Prioridad en el cliente: Satisfacer al cliente mediante entregas tempranas y continuas de software de valor.
2. Bienvenidos los requerimientos cambiantes: Aceptar los cambios incluso en etapas avanzadas del desarrollo para ventaja competitiva.
3. Entregas frecuentes de software funcional: Entregar código operativo en ciclos cortos (desde 2 semanas a 2 meses).
4. Trabajo conjunto diario: Los responsables del negocio y los desarrolladores deben trabajar juntos día a día.
5. Individuos motivados: Construir proyectos con personas motivadas, dándoles el entorno, el apoyo y la confianza necesarios.

6. Comunicación cara a cara: El diálogo directo e interactivo es el método más eficiente de transmitir información.
7. Medida principal de progreso: El software funcionando es el indicador real del avance.
8. Desarrollo sostenible: Mantener un ritmo de trabajo constante e indefinido para evitar el agotamiento del equipo
9. Excelencia técnica: La atención continua al buen diseño y la calidad técnica mejora la agilidad
10. Simplicidad esencial: La simplicidad es el arte de maximizar la cantidad de trabajo no realizado (no construir lo que no fue pedido)
11. Equipos autoorganizados: Las mejores arquitecturas, requerimientos y diseños emergen de equipos con autonomía
12. Mejora continua (Reflexión regular): A intervalos regulares, el equipo reflexiona sobre cómo ser más efectivo y ajusta su conducta

Entonces la filosofía ágil tiene como objetivo entregar software de forma frecuente en un entorno cambiante. Se define como un compromiso entre nada de proceso y demasiado proceso. El agilismo no se apoya en procesos deterministas, sino en el control empírico de procesos, el cual sostiene que el conocimiento proviene de la experiencia y la toma de decisiones se basa en lo que se observa. El empirismo se sostiene en 3 pilares fundamentales:

1. Transparencia: todos los aspectos del proceso y del trabajo emergente deben ser totalmente visibles para quienes ejecutan el trabajo y quienes reciben el producto. La falta de transparencia oculta riesgos y conduce a decisiones erróneas.
2. Inspección: el progreso hacia los objetivos y los artefactos debe inspeccionarse con frecuencia y diligencia para detectar variaciones o defectos indeseables (INSPECCIONAR SIN ADAPTAR = INÚTIL)
3. Adaptación: si se detecta que un aspecto del proceso se desvía o que el producto es inaceptable se debe realizar un ajuste lo antes posible para minimizar desviaciones.

Filosofía Lean: proviene del sistema de producción de Toyota de los años 40. Fue adaptada por Mary y Tom Poppendieck (*Lean Software Development*). Tiene como premisa central maximizar el valor entregado al cliente con el mínimo consumo de recursos, bajo la convicción de que todo esfuerzo o artefacto que no genere valor directo para el cliente se considera desperdicio. La idea es producir exactamente lo que el cliente final quiere, cuando lo quiere, donde lo quiere, a un costo mínimo y un precio justo. Para esto utiliza el método Just inTime, que plantea que cada fase del desarrollo procese y entregue sus artefactos justo en el momento en que se necesitan, en la cantidad justa y con la información adecuada, bajo la premisa de "analice cuando lo necesite, no antes". Para hacer efectivo el JIT, Lean reemplaza el modelo tradicional de "empujar" trabajo (*Push*) por un sistema de arrastre (Pull), en el cual los desarrolladores "tiran" del trabajo disponible del *backlog* únicamente cuando cuentan con la capacidad operativa para procesarlo. Se adaptaron los principios industriales de Toyota a la intangibilidad del desarrollo de software en 7 principios:

1. Eliminar los desperdicios: Todo paso, actividad o artefacto que no añada valor directo para el usuario debe ser detectado y erradicado.
2. Amplificar el aprendizaje: Fomentar la experimentación y los ciclos cortos de retroalimentación para convertir el conocimiento tácito e individual en explícito para todo el equipo.
3. Diferir compromisos (decidir lo más tarde posible): postergar las decisiones irreversibles hasta el último momento responsable, maximizando la información disponible y evitando especificaciones por adelantado.
4. Entregar lo antes posible: reducir el tiempo de ciclo para entregar incrementos pequeños de valor antes de que las necesidades del negocio o del contexto cambien.
5. Dar poder al equipo (potenciar las personas): capacitar y brindar autonomía a equipos multifuncionales autoorganizados, confiando en sus decisiones técnicas.
6. Embeber la integridad conceptual (calidad intrínseca): garantizar que el software se perciba como una solución coherente, armónica y libre de defectos desde su construcción.
7. Ver el todo (visión holística): Optimizar la cadena de valor completa del producto en lugar de suboptimizar áreas o departamentos aislados.

Los 7(+1) desperdicios de Lean para software: En el desarrollo de software, la materia prima principal es el conocimiento abstracto. Por ello, los desperdicios industriales (*Muda*) se reinterpretan adaptándolos a los servicios basados en conocimiento:

1. Características extra (Over-production): Desarrollar funcionalidades no solicitadas o con baja frecuencia de uso, recordando que la mayor parte del valor reside en una fracción mínima de características.
2. Trabajo parcialmente hecho (Inventory): Acumulación de historias de usuario, especificaciones o código a medias sin integrar ni probar, el cual se degrada con el tiempo.
3. Procesos extra (Extra Processing): Pasos burocráticos, documentación superflua o reprobaciones que no agregan valor al producto.
4. Reaprendizaje (Re-learning): Pérdida de conocimiento acumulado por mala comunicación que obliga a investigar o resolver el mismo problema varias veces.
5. Transferencia de conocimiento / Handoffs (Transportation): Pasajes de información entre personas o áreas desconectadas que generan filtraciones o distorsiones de la idea original.
6. Esperas (Waiting): Tiempos muertos esperando definiciones, aprobaciones o disponibilidad de entornos de prueba.
7. Cambios de tarea / Multitasking (Motion): Interrupciones y saltos constantes entre tareas que exigen tiempos de reconfiguración mental y reducen la productividad.
8. Defectos (Defects): Errores que generan retrabajo y consumen capacidad operativa que podría destinarse a nuevas funcionalidades de valor.

Relación Lean-Ágil: son enfoques complementarios que comparten una base empírica: aceptan que el desarrollo de software tiene incertidumbre y que, por lo tanto, es necesario trabajar mediante transparencia, inspección, retroalimentación y adaptación continua. Lean surge del Sistema de Producción de Toyota y posteriormente se adapta al desarrollo de software, mientras que el Agilismo se consolida con el Manifiesto Ágil de 2001 como respuesta a la rigidez de los modelos tradicionales.

Ambos buscan maximizar el valor entregado al cliente y reducir el trabajo que no aporta valor. Comparten principios como la eliminación de desperdicios, la adaptación a cambios, las decisiones tomadas en el momento adecuado (Just-in-Time), los sistemas Pull, donde se toma nuevo trabajo según la capacidad disponible, y el respeto por equipos autónomos y multifuncionales.

La principal diferencia está en el nivel donde ponen el foco. Ágil se orienta principalmente a la organización del equipo y a la entrega frecuente de software funcional, utilizando marcos como Scrum y sus iteraciones o sprints. Lean, en cambio, aporta una visión más global del proceso y busca optimizar el flujo de trabajo completo, reducir tiempos y eliminar cuellos de botella. Herramientas como Kanban permiten visualizar ese flujo y limitar el trabajo en progreso (WIP). Por eso, Lean y Ágil se complementan: Ágil favorece la adaptación y las entregas frecuentes, mientras Lean busca que todo ese proceso sea eficiente y genere el máximo valor posible.

14. REQUERIMIENTOS ÁGILES

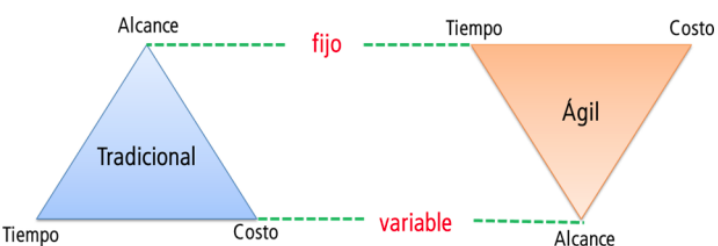
Requerimientos Ágiles: representan un cambio de enfoque frente a la ingeniería de requerimientos tradicional: se pasa de intentar especificar el 100% del sistema por adelantado (*Big Requirements Up Front - BRUF*, lo cual genera desperdicio ya que gran parte del software no se termina utilizando) a descubrirlos y refinarlos de forma continua y emergente a lo largo del desarrollo. Se gestionan mediante un contenedor dinámico y transitorio llamado Product Backlog, que es priorizado por el PO según el valor que aporta al negocio. La gestión de requerimientos en ambientes Lean-Ágiles se sostiene en 3 pilares fundamentales:

1. Usar el valor para construir el producto correcto: el foco de la gestión ágil está en construir una solución que resuelva un problema real del cliente y le genere valor al negocio, en lugar de entregar únicamente ejecutables o productos que no satisfacen necesidades reales.
2. Determinar qué es “solo lo suficiente”: los requerimientos deben ser encontrados de a poco, empezando por lo mínimo necesario para generar retroalimentación, alimentar el empirismo e ir descubriendo el detalle conforme se avanza.
3. Usar historias y modelos para mostrar que construir: se basa en el principio de “técnicos y no técnicos trabajando juntos”, utilizando la comunicación personal y técnicas de descubrimiento como las US para construir la visión del producto junto al referente de negocio.

La Ingeniería de Requerimientos (IR) es la disciplina de la ingeniería de software que abarca el proceso sistemático de descubrir, analizar, documentar, verificar y gestionar los servicios que un sistema debe proporcionar y las restricciones bajo las cuales debe operar. Representa el puente crítico entre las necesidades del negocio/cliente y el diseño e implementación técnica del software. Se divide estructuralmente en dos procesos complementarios:

- El desarrollo de requerimientos comprende las actividades necesarias para determinar qué debe construirse. Incluye la elicitación, donde se descubren las necesidades de los usuarios; el análisis y negociación, para resolver conflictos y establecer prioridades; la especificación, donde los requerimientos se documentan y modelan; y la validación, que comprueba que sean completos, consistentes, realistas y verificables antes de desarrollar el producto.
- La gestión de requerimientos se ocupa de mantener esos requerimientos controlados y consistentes durante todo el ciclo de vida. Incluye el control de cambios, evaluando el impacto y costo de cada modificación, y la trazabilidad bidireccional, que permite relacionar cada requerimiento con su origen, diseño, código y pruebas, y también realizar el recorrido inverso.

La triple restricción: el equilibrio de un proyecto se administra mediante las variables de Alcance, Tiempo y Costo (Recursos), manteniendo la Calidad en el centro:



- Enfoque Tradicional: El Alcance se fija al inicio en función de las necesidades percibidas del cliente, mientras que el Tiempo y el Costo se estiman y derivan de este. Si se incorporan cambios en el alcance, se modifican los recursos y el tiempo, generando sobrecostos o retrasos.
- Enfoque Ágil: Se invierte el triángulo de gestión: se fijan el Tiempo (mediante *sprints* o *timeboxes* de 1 a 4 semanas) y los Recursos/Costo (equipo de trabajo fijo), dejando el Alcance como la variable flexible que se adapta continuamente para entregar el máximo valor al negocio.

BRUF (Big Requirements Up Front): es una práctica tradicional que busca especificar el 100 % de los requerimientos antes de comenzar el desarrollo. El principal problema es que genera desperdicio: una parte importante de las funcionalidades definidas inicialmente puede terminar sin utilizarse o utilizarse muy poco. Además, intentar mantener una especificación fija frente a requerimientos cambiantes provoca reprocesamiento y mayores costos.

Product Backlog (Pila de Producto): lista emergente, ordenada y dinámica que contiene las características, funcionalidades y mejoras necesarias para el producto, y constituye la fuente de trabajo del equipo. Su gestión es responsabilidad del Product Owner (PO), quien ordena los ítems según su valor para el negocio. A diferencia de una especificación estática, el backlog nunca está completamente terminado, ya que sus elementos pueden agregarse, eliminarse o reordenarse según la retroalimentación. Cuando un ítem se implementa, su valor queda incorporado en el producto funcional y sus casos de prueba.

Requerimientos Just in Time (JiT): se basan en principios Lean como eliminar desperdicios y diferir decisiones hasta el último momento responsable. El proyecto comienza con una visión general y los requerimientos mínimos necesarios, mientras que el resto se descubre, analiza y detalla progresivamente, justo antes de ser seleccionado para su implementación en las próximas iteraciones.

Fuente de Requerimientos: En el enfoque tradicional, la principal fuente de requerimientos es la ERS (Especificación de Requerimientos de Software), un documento formal que el cliente debe revisar y validar. En el enfoque ágil, la fuente principal es la comunicación continua entre el equipo y el cliente o Product Owner. La menor documentación formal se compensa con una interacción frecuente que permite comprender y ajustar las necesidades durante el desarrollo.

Tipos de Requerimientos: Los requerimientos pueden organizarse en dos grandes dominios.

El Dominio del Problema representa lo que necesita el cliente y el negocio. Incluye los requerimientos de negocio, que expresan objetivos generales de la organización, y los requerimientos de usuario, que describen los servicios que el usuario espera recibir. Las User Stories son una forma de expresar requerimientos de usuario vinculados con una necesidad de negocio.

El Dominio de la Solución corresponde principalmente al equipo técnico e incluye los requerimientos de software. Los funcionales indican qué debe hacer el sistema; los no funcionales establecen cómo debe funcionar, incluyendo restricciones de calidad, rendimiento, seguridad o normativas; y los requerimientos de implementación establecen restricciones relacionadas con la infraestructura y las tecnologías utilizadas para desplegar la solución.

15.USER STORIES

User Stories: son la técnica principal de la Ingeniería de Requerimientos Ágiles para capturar necesidades de los usuarios (no a nivel sistema) expresadas en términos de valor de negocio. No son especificaciones técnicas de software ni contratos estáticos exhaustivos, sino mecanismos para diferir una conversación entre el *Product Owner* y el equipo de desarrollo hasta el momento justo antes de su implementación (*Just in Time*).

Representan porciones verticales de funcionalidad, lo que significa que atraviesan todas las capas de la arquitectura de software (Front-Back-BD) para entregar un incremento de producto terminado y utilizable.

Modelo de las 3 C: una User Story se construye a partir de tres componentes fundamentales: Tarjeta (Card), Conversación (Conversation) y Confirmación (Confirmation).

1. Tarjeta (Card): es el registro físico o digital que funciona como recordatorio de la funcionalidad. Incluye una frase verbal en infinitivo que identifica brevemente la acción, como *Comprar entradas* o *Retirar paquete*. La User Story se redacta utilizando la siguiente estructura:

Como «rol» yo puedo «actividad» de forma tal que «valor de negocio».

El rol indica quién realiza la acción o recibe el beneficio y nunca debe ser “*el sistema*”. La actividad describe qué acción funcional se realizará y el valor de negocio explica para qué sirve o qué beneficio obtiene el usuario, ayudando al Product Owner a establecer prioridades. La tarjeta también contiene los criterios de aceptación, que establecen las reglas y límites que debe cumplir la funcionalidad. Se redactan a un alto nivel de abstracción, sin entrar en detalles técnicos de implementación, y permiten determinar cuándo la historia puede considerarse correctamente desarrollada.

2. Conversación (Conversation): la parte más importante de una User Story. Consiste en la comunicación entre el Product Owner y los desarrolladores para aclarar dudas, comprender las necesidades, negociar detalles y definir correctamente el alcance de la funcionalidad. La tarjeta funciona como recordatorio, pero es mediante la conversación que el equipo obtiene el conocimiento necesario para desarrollarla.

3. Confirmación (Confirmation): consiste en definir las pruebas de aceptación que permitirán comprobar que la funcionalidad fue implementada de manera correcta y completa. Las pruebas pueden redactarse como:

Probar [acción/datos ingresados] → pasa / falla.

Deben incluir tanto casos exitosos o “camino feliz” como situaciones de error, por ejemplo datos inválidos, datos faltantes o límites superados. De esta manera, los criterios indican qué debe cumplirse y las pruebas permiten comprobar que efectivamente se cumple.

La Definition of Ready (DoR) funciona como un filtro de entrada que determina si una Historia de Usuario o ítem del *backlog* está lo suficientemente claro, desglosado y comprendido como para ingresar a un *Sprint* de desarrollo. En la gestión de proyectos se considera una cualidad emergente del refinamiento continuo —y no un compromiso formal estático— cuyo propósito es mitigar riesgos antes de escribir código. Un ítem alcanza el estado *Ready* cuando satisface el modelo de calidad INVEST (es independiente, negociable, valioso, estimable, pequeño y testeable), posee sus criterios de aceptación definidos y ha resuelto cualquier incertidumbre técnica o funcional crítica previa.

Por otro lado, la Definition of Done (DoD) actúa como un filtro de salida y representa el compromiso formal asociado al *Incremento* en Scrum, definiendo los estándares indispensables para dar por finalizada la construcción. Se gestiona de forma binaria —sin admitir porcentajes parciales de avance— y se divide en dos dimensiones complementarias: el Output Done (calidad técnica colectiva de los desarrolladores mediante revisión de código, pruebas integradas, CI/CD y cero defectos) y el Outcome Done (validación del valor real del producto y negocio evaluada por el *Product Owner* a través del comportamiento de usuario y telemetría).

Modelo de Calidad INVEST (Definition of Ready - DoR): Para que una User Story se considere “Lista” (*Ready*) para ingresar a un *Sprint* o iteración de desarrollo, debe cumplir con las seis propiedades del modelo INVEST:

- I – Independent (Independiente): Se puede implementar y calendarizar en cualquier orden sin depender fuertemente de otras historias.
- N – Negotiable (Negociable): No es un contrato cerrado; expresa una intención cuyo “cómo” se negocia durante la conversación.
- V – Valuable (Valiosa): Aporta un valor explícito de negocio para el usuario o cliente final.

- E – Estimatable (Estimable): Cuenta con suficiente claridad para que el equipo pueda evaluar su complejidad, esfuerzo e incertidumbre.
- S – Small (Pequeña / Tamaño adecuado): Es lo suficientemente acotada como para completarse e integrarse al 100 % dentro de una sola iteración.
- T – Testable (Comprobable / Testable): Se pueden derivar pruebas de aceptación objetivas que permitan verificar su cumplimiento.

Niveles de Abstracción: User Stories, Épicas y Temas:

- Una User Story es un ítem de tamaño adecuado que cumple con el criterio de *Ready*, por lo tanto, se encuentra en condiciones de ingresar a una iteración para ser desarrollada.
- Una Épica es una historia de gran tamaño que excede la capacidad de una iteración o que todavía no cumple con el modelo INVEST. Por esta razón, debe dividirse obligatoriamente en User Stories más pequeñas antes de poder ser planificada dentro de una iteración.
- Los Temas (Themes) son agrupaciones conceptuales de varias User Stories relacionadas. Permiten tratar un conjunto de historias vinculadas como una entidad simple al momento de organizar y planificar el trabajo.

User Story Canónica: es una historia de referencia dentro del *Product Backlog* que presenta la menor complejidad y una incertidumbre prácticamente nula. Funciona como un “metro patrón” para realizar estimaciones relativas de las demás historias. A esta historia se le asigna normalmente un valor pequeño, como 1 o 2 Story Points, y posteriormente las demás User Stories se comparan con ella para determinar cuánto más complejas, grandes o inciertas resultan.

Spike: tipo especial de Historia de Usuario utilizado para reducir el riesgo y la incertidumbre técnica o funcional de una funcionalidad o del proyecto. Cuando existe un grado elevado de incertidumbre, el requerimiento todavía no cumple con la condición de *Ready* necesaria para ingresar directamente al desarrollo. Por este motivo, primero se realiza una investigación mediante un Spike. Una vez resuelta la investigación, el conocimiento obtenido permite definir con mayor precisión el trabajo necesario. Como resultado, el Spike puede dar origen a una o más User Stories correctamente definidas y estimables.

Spikes Técnicos: se enfocan en investigar aspectos relacionados con el dominio de la solución y la implementación. Se utilizan, por ejemplo, para evaluar la viabilidad de una nueva tecnología o API, analizar el rendimiento potencial de una alternativa, tomar decisiones de “hacer vs. comprar” o comprobar determinadas limitaciones técnicas antes de comprometerse con una solución.

Spikes Funcionales: se utilizan cuando existe incertidumbre acerca de la interacción entre el usuario y el sistema necesaria para obtener el beneficio esperado. Habitualmente se resuelven mediante el desarrollo de prototipos o mockups, que permiten obtener retroalimentación directa de los usuarios o del *Product Owner* antes de definir completamente las User Stories correspondientes.

Lineamientos para el uso de Spikes: deben ser estimables, demostrables y aceptables. Además, representan una excepción y no la regla, ya que el equipo debe aprender a trabajar con cierto grado de riesgo e incertidumbre dentro de cada iteración. Por este motivo, deberían utilizarse principalmente cuando existen incógnitas importantes o críticas que impiden definir correctamente una funcionalidad.

En general, los Spikes deben implementarse en un *Sprint* o iteración separada de las User Stories que surjan como resultado de la investigación. La excepción ocurre cuando el Spike es muy pequeño y puede resolverse rápidamente, en cuyo caso puede realizarse dentro de la misma iteración.

Investment Themes (Temas de Inversión): representan un conjunto de iniciativas o propuestas de valor que orientan la inversión estratégica de una organización en sistemas, productos y servicios. Su objetivo principal es contribuir a la diferenciación en el mercado y permitir que la organización obtenga ventajas competitivas. Estos temas permiten que la alta gerencia y el *Product Owner* puedan priorizar y distribuir el presupuesto y el *Product Backlog*, buscando un equilibrio entre diferentes horizontes de inversión.

- La inversión en ofertas existentes comprende las mejoras, el soporte y el mantenimiento de productos que ya se encuentran en producción. Su objetivo es sostener y mejorar las soluciones que actualmente generan valor para la organización.
- La inversión en nuevos productos o servicios a corto plazo comprende funcionalidades y soluciones destinadas a incrementar los beneficios o ganar participación de mercado durante el período actual o en un futuro cercano.
- La inversión a futuro se orienta al desarrollo de productos y servicios más avanzados que permitan posicionar estratégicamente a la empresa en el mediano plazo.
- La inversión de largo plazo comprende iniciativas cuyos beneficios o retorno de inversión se esperan recién después de varios años, por lo que representan apuestas estratégicas de mayor horizonte temporal.

16. ESTIMACIONES ÁGILES

Estimación: proceso de obtener una predicción analítica e imparcial sobre una medida (trabajo/tamaño) con el objetivo de evaluar la completitud del producto y gestionar los riesgos del proyecto. Es una predicción de cuánto tiempo durará o costará un desarrollo, siendo una de las actividades más complejas de la disciplina. Las estimaciones no son planes, ni compromisos, ES PREDECIR UN ESCENARIO PROBABILISTICO A FUTURO, mientras que planificar implica organizar la ejecución asumiendo riesgos específicos. Las estimaciones sirven de insumo para elaborar los planes, pero el plan final no tiene por qué coincidir con el valor estimado.

Los errores y desviaciones en las estimaciones vienen de diversas fuentes. En primer lugar tenemos que la información puede ser imprecisa o puede haber imprecisión en el proceso (es decir, poca claridad en el alcance inicial o sobre la capacidad del equipo), también encontramos la omisión de actividades clave, que es el error más común que se da cuando se olvidan tareas necesarias como la elaboración de documentación, datos de pruebas y participación en revisiones técnicas, mantenimiento, etc. También encontramos el “cono de la incertidumbre” y la estimación prematura, que consiste en pretender una exactitud absoluta al inicio del proyecto.

Por sobre todas las cosas encontramos la confusión entre estimación y compromiso. Esto se debe a que se trata a la estimación como una promesa fija, lo que lleva a subestimar (presionar/justificar el valor artificialmente hacia abajo por miedo al rechazo del cliente, obligando a rechazar etapas de análisis/diseño y preparar soluciones quick'n dirty) y a sobreestimar (consiste en inflar las horas por temor a imprevistos, generando desperdicio e ineficiencia en la gestión).

Se lleva adelante el proceso de estimación por dos razones importantes de gestión:

1. Para predecir completitud y factibilidad: permite obtener una respuesta temprana para ver si el trabajo es viable.
2. Para administrar y mitigar riesgos: protección para el equipo de desarrollo, permitiendo tomar decisiones de control a tiempo.

El mayor beneficio de la estimación radica en hacerla. La actividad reúne al equipo, enriquece el entendimiento de los problemas y permite descubrir incertidumbre o malentendidos.

Métodos utilizados para estimar:

A)- Basados en Experiencia:

- Datos Historicos: Consisten en recopilar métricas de proyectos pasados (de la industria, de la organización o del propio proyecto) para transferir el conocimiento a la empresa y evitar depender de la memoria de una persona.
- Juicio Experto Puro: Un profesional evalúa las especificaciones basándose en su conocimiento acumulado. Presenta el riesgo de que la capacidad de estimación se pierda si el experto se retira de la organización.
- Juicio Experto Wideband Delphi: Método grupal e iterativo conducido por un coordinador anónimo/moderado donde varios expertos realizan estimaciones individuales, discuten supuestos y recalculan hasta alcanzar un acuerdo. Es el antecedente directo del Poker Planning.

B)- Métodos por analogía: consisten en comparar de forma relativa los factores del nuevo producto (tamaño, complejidad, cantidad de usuarios) con proyectos o módulos similares desarrollados con anterioridad.

C)- Otros métodos:

- Basados exclusivamente en recursos: evalúan únicamente el personal disponible y adaptan el alcance a la capacidad.
- Basados exclusivamente en el mercado: fechas y presupuestos vienen dados por la competencias o clientes.
- Basados en componentes o el proceso de desarrollo: puede ser Top-Down (descomposición inicial del producto en grandes bloques funcionalidades) y Bottom-Up (descomposición del producto en unidades elementales lo mas pequeñas posibles para estimar individualmente y luego sumar)
- Métodos algorítmicos: Utilizan fórmulas matemáticas empíricas parametrizadas por variables de entrada como líneas de código, puntos de función o el modelo COCOMO II.

Estimaciones en ambientes ágiles: En agilidad, la estimación deja de ser un cálculo estático para convertirse en una presunción numérica probabilística.

Rechazo de la Precisión Absoluta: Buscar una precisión microscópica al inicio es considerado un desperdicio (*Muda* en Lean), dado que “la precisión es cara” y el costo del error inicial no se elimina invirtiendo más tiempo en el análisis *Up-Front*.

El Valor del Proceso: El mayor beneficio del proceso de estimación radica en hacer la estimación en sí misma, ya que la discusión en grupo permite comprender el dominio, alinear criterios y detectar riesgos de forma temprana.

Estimaciones Relativas (Por Comparación): Las personas son ineficientes determinando magnitudes absolutas en horas, pero son mucho más precisas comparando tamaños entre elementos.

¿Quién Estima?: El propio **equipo de desarrollo** que ejecutará las tareas. Delegar la estimación a un líder o experto ajeno al trabajo diario es una de las principales causas de desvío en los proyectos tradicionales.

¿Qué se Estima? En ambientes ágiles se estima únicamente el **TAMAÑO** relativo de las Historias de Usuario (*User Stories / Features*), expresado en Story Points. No se estiman directamente horas de trabajo ni calendarios para la totalidad del proyecto, ya que la duración de las iteraciones es fija (*timebox*).

Tamaño vs. Esfuerzo: El Tamaño es una medida objetiva e independiente de quién realice la tarea. Integra tres variables: Complejidad + Esfuerzo + Incertidumbre/Riesgo. Para el equipo, el tamaño de una funcionalidad es el mismo sin importar quién programe la tarea. El Esfuerzo representa las horas humanas lineales de trabajo efectivo dedicadas al desarrollo de la historia. Es subjetivo para cada persona, ya que depende directamente de sus habilidades, experiencia, conocimientos y familiaridad con el dominio. Es importante diferenciar Esfuerzo y Calendario. El esfuerzo mide las horas de trabajo lineal efectivo, mientras que el tiempo de calendario representa los días reales transcurridos e incluye reuniones, pausas, soporte e imprevistos. Por lo tanto, una jornada de 8 horas no equivale necesariamente a 8 horas de esfuerzo lineal continuo.

Velocidad (Velocity): La Velocidad es una métrica empírica del progreso del equipo: se calcula, no se estima. Se obtiene sumando los Story Points de las User Stories 100 % completadas y aceptadas por el Product Owner al finalizar una iteración, siempre que cumplan con la Definition of Done. La gestión es binaria: si una historia queda parcialmente terminada, por ejemplo al 90 %, aporta 0 Story Points a la velocidad de esa iteración. Además, la velocidad posee un efecto de autocorrección. Al calcular el promedio a lo largo de varios *Sprints*, las diferencias de las estimaciones iniciales se absorben y corrigen de manera automática.

Reestimación: Una User Story debe reestimarse solamente cuando cambia la percepción de su tamaño relativo respecto de las demás historias del *backlog*. El objetivo es mantener la consistencia del patrón utilizado para comparar las historias. No se debe reestimar una historia simplemente porque su implementación haya requerido más o menos tiempo del esperado. Las variaciones individuales de tiempo se equilibran mediante la velocidad del equipo a lo largo de las distintas iteraciones.

Planificación en Múltiples Niveles: Los proyectos ágiles aplican la inversión de la triple restricción: mantienen fijos el Tiempo, mediante la duración de las iteraciones, y los Recursos, mientras que el Alcance permanece como la variable adaptable. Existen cinco horizontes de planificación:

- Portfolio / Estrategia (1 año o más): Se ocupa de la selección y administración del *backlog* de productos de la organización.
- Producto (meses o años): Define la visión global del producto y su *roadmap* de evolución.
- Release (3 a 9 meses): Planifica un conjunto de User Stories u Opciones Comercializables (MMF) que serán entregadas al mercado. Su duración puede calcularse mediante:
 - Duración en Sprints = Total de Story Points del Product Backlog / Velocidad Promedio del Equipo
- Iteración / Sprint Planning (1 a 4 semanas): Se selecciona el conjunto prioritario de historias del Product Backlog que integrará el Sprint Backlog para alcanzar el *Sprint Goal*.
- Día / Daily Scrum (diario): Se realiza la coordinación, sincronización e inspección del trabajo correspondiente a las últimas 24 horas.

Story Points (Puntos de Historia): es una unidad de medida abstracta, homogénea y propia de cada equipo que representa el peso relativo de una User Story. Los Story Points consideran tres componentes: Complejidad + Esfuerzo + Incertidumbre/Riesgo. Entre las escalas utilizadas se encuentran la serie de Fibonacci modificada (0, ½, 1, 2, 3, 5, 8, 13, 20, 40, 100), los talles de remeras (S, M, L, XL) y la serie 2ⁿ. Según la Regla del Metro Patrón, una vez acordada una escala para un equipo, esta no debe modificarse, ya que hacerlo destruiría la base comparativa histórica utilizada para realizar las estimaciones.

La User Story Canónica es la historia de referencia del *backlog* que presenta la menor complejidad e incertidumbre nula. Generalmente se le asignan 1 o 2 Story Points y funciona como “metro patrón” para estimar relativamente las demás historias.

Poker Planning (Poker Estimation): es una técnica de estimación colaborativa ampliamente utilizada en la práctica ágil. Combina juicio experto, analogía y desagregación, aprovechando la “sabiduría colectiva” del equipo. Participa todo el equipo de desarrollo (Developers). El Product Owner se encarga de aclarar dudas relacionadas con el alcance y el negocio, pero no estima ni vota. La dinámica comienza cuando el Product Owner presenta la User Story y se leen sus criterios de aceptación. Luego, el equipo aclara las dudas existentes y toma como referencia la User Story Canónica. Cada participante selecciona en privado una carta de su mazo con la estimación que considera adecuada. Todas las cartas se revelan simultáneamente para evitar el efecto de anclaje o la influencia del orden jerárquico. Si existen diferencias importantes, los participantes que realizaron las estimaciones extremas —la más alta y la más baja— explican sus argumentos y los riesgos que identificaron. El equipo debate los distintos supuestos y posteriormente vuelve a votar. El proceso se repite hasta alcanzar un consenso sobre la estimación.

17.FRAMEWORK SCRUM

Scrum: es un marco de trabajo (framework) liviano y adaptativo diseñado para ayudar a personas, equipos y organizaciones a generar valor a través de soluciones para problemas complejos. No es una metodología prescriptiva ni un proceso paso a paso, sino una estructura genérica basada en un conjunto mínimo de roles, eventos, artefactos y reglas.

Pilares de Scrum: Scrum se fundamenta en el control empírico de procesos y el pensamiento Lean:

- Transparencia: El proceso y el trabajo emergente deben ser totalmente visibles tanto para quienes realizan el trabajo como para quienes reciben el resultado.
- Inspección: El progreso hacia las metas y los artefactos se evalúa con frecuencia para detectar variaciones o defectos indeseados.
- Adaptación: Si se detectan desvíos en el proceso o producto, se realiza un ajuste inmediato. *Inspeccionar sin adaptar se considera un desperdicio.*

El Equipo Scrum (Scrum Team - Ecosistema SGEP 2026): Es un equipo pequeño, multifuncional y autogestionado. En la evolución del marco (SGEP 2026), el ecosistema de roles se estructura de la siguiente manera:

- Product Owner (Humano): *Accountable* de maximizar el valor del producto y de gestionar eficazmente el *Product Backlog*.
- Scrum Master (Humano): *Accountable* de la efectividad del equipo y de liderar la adopción de Scrum como líder servicial.
- Product Developers (Humanos + Inteligencia Aumentada): Responsables de crear un Incremento utilizable en cada Sprint. En SGEP 2026, la IA se integra como un socio/compañero supervisado (*human-in-the-loop*), manteniendo la responsabilidad final en el profesional humano.
- Roles de Soporte (Supporters y Stakeholders): Los *Stakeholders* son los interesados o clientes del producto, mientras que los *Supporters* son patrocinadores organizacionales que eliminan barreras de entorno para sostener la cultura ágil.

Eventos de Scrum: Son bloques de tiempo acotados (*timeboxes*) diseñados para permitir la transparencia, inspección y adaptación.

- Sprint (Contenedor): Iteración de 1 mes o menos (usualmente de 2 a 4 semanas) donde las ideas se transforman en valor.
- Sprint Planning: Diseña el plan de trabajo respondiendo al ¿Por qué?, ¿Qué? y ¿Cómo?. Su *timebox* es de máximo 8 horas para un Sprint de 1 mes.
- Daily Scrum: Sincronización diaria de 15 minutos para los Developers, donde se evalúa el progreso hacia el *Sprint Goal* y se ajusta el plan de las siguientes 24 horas.

- **Sprint Review:** Inspección colaborativa del Incremento al final del Sprint junto con los *stakeholders*. Su *timebox* es de máximo 4 horas para un Sprint de 1 mes.
- **Sprint Retrospective:** Evento de cierre donde el equipo reflexiona e inspecciona personas, procesos y herramientas para acordar mejoras continuas. Su *timebox* es de máximo 3 horas para un Sprint de 1 mes.
- **Refinamiento del Product Backlog:** Actividad continua, no un evento estático, para descomponer y dar claridad a las Historias de Usuario. Consume habitualmente hasta el 10 % del tiempo del Sprint.

Artefactos y sus Compromisos Formales: Cada artefacto contiene un compromiso obligatorio que garantiza la transparencia y permite medir el progreso:

- **Product Backlog:** Lista ordenada y emergente de todo lo que se requiere para mejorar el producto. Compromiso: Product Goal (Objetivo del Producto).
- **Sprint Backlog:** Plan táctico que incluye el *Sprint Goal*, los ítems seleccionados y el plan de ejecución para la iteración. Compromiso: Sprint Goal (Objetivo del Sprint).
- **Incremento:** La suma de todos los ítems completados en el Sprint que cumplen las medidas de calidad. Compromiso: Definition of Done (DoD). En SGEP 2026 se desglosa en Definition of Output Done para la calidad técnica del código y Definition of Outcome Done para la validación del impacto de valor.

Product Goal (Objetivo del Producto): Es el compromiso formal del Product Backlog. Representa la meta o dirección estratégica a largo plazo para el equipo Scrum. El equipo debe cumplir o abandonar un objetivo antes de asumir el siguiente, funcionando como el “paso intermedio” o corte vertical tangible de la visión del producto.

Timebox (Bloque de Tiempo): Es la fijación de un tiempo máximo innegociable para la ejecución de cada evento de Scrum. Si al finalizar el *timebox* el trabajo no se ha completado en su totalidad, no se extiende el tiempo; en su lugar, se reduce el alcance entregado. Esto promueve un ritmo de trabajo constante y sostenible, volviendo predecible la cadencia del equipo.

Capacidad (Capacity): Es una métrica que utiliza Scrum para determinar cuánta cantidad de trabajo puede comprometer un equipo en un Sprint específico. Se estima en el *Sprint Planning* calculando las horas disponibles de cada desarrollador y descontando ausencias, un *buffer* de imprevistos, tareas externas (soporte/mantenimiento) y el tiempo dedicado a los eventos de Scrum. Para equipos maduros, la capacidad se suele estimar en Story Points, mientras que para equipos menos maduros se puede medir en horas ideales.

Niveles de Planificación: La planificación ágil se organiza en 5 horizontes temporales:

Nivel	Horizonte	Participantes	Foco Principal	Entregable / Artefacto
Portfolio	1 año o más	Stakeholders y Product Owners	Administración de un Portfolio de Productos	Backlog de Portfolio
Producto	Varios meses o más	Product Owner y Stakeholders	Visión y evolución del producto en el tiempo	Visión de Producto, Roadmap y Características
Release	3 a 9 meses	Equipo Scrum entero y Stakeholders	Balancear valor, calidad, alcance, cronograma y costo	Plan de Release
Iteración	1 a 4 semanas	Equipo Scrum entero	Definir qué aspectos entregar en el próximo Sprint	Sprint Goal y Sprint Backlog
Día	Diaria	Scrum Master y Developers	Inspeccionar el progreso y planificar las siguientes 24 hs	Inspección del avance

Herramientas de Scrum: Las herramientas visuales más destacadas en el marco incluyen:

- Tarjeta de Tarea (Task Card): Representación de una unidad de trabajo que consta de 5 partes fundamentales:
- Código del Ítem de Backlog: Identificador del PBI de origen.
- Nombre de la Actividad: Frase verbal que especifica qué hay que hacer.
- Iniciales del Responsable: Persona a cargo de la tarea.
- Número de Día del Sprint: Día en que se asigna o ejecuta.
- Tiempo Estimado: Horas necesarias para finalizarla.

Tablero de Tareas (Taskboard): Espacio donde se mueven las tarjetas para reflejar el estado del Sprint. Se organiza habitualmente en 5 columnas: *Story* (nombre de la historia), *To Do* (pendientes), *Work In Process* (en curso), *To Verify* (finalizadas a verificar) y *Done* (terminadas).

Gráficos del Backlog (Burn Charts):

- **Sprint Burn-Down Chart:** Gráfico con pendiente descendente que muestra el **trabajo restante** (en horas o Story Points) a lo largo de los días del Sprint.
- **Sprint Burn-Up Chart:** Gráfico con pendiente ascendente que muestra la **acumulación de trabajo completado** a lo largo de la *release*.

- **Combined Burn Chart:** Combina en un solo diagrama el trabajo restante y el trabajo completado.

MVP — Producto Mínimo Viable: Su objetivo es aprender y validar hipótesis de negocio con el menor esfuerzo posible, probando el producto principalmente con early adopters. Busca comprobar si los clientes realmente necesitan la solución antes de invertir demasiado en desarrollarla. Puede ir desde una maqueta o prueba de humo hasta un prototipo funcional con las características mínimas necesarias y la propuesta de valor. MVP = Aprender y validar.

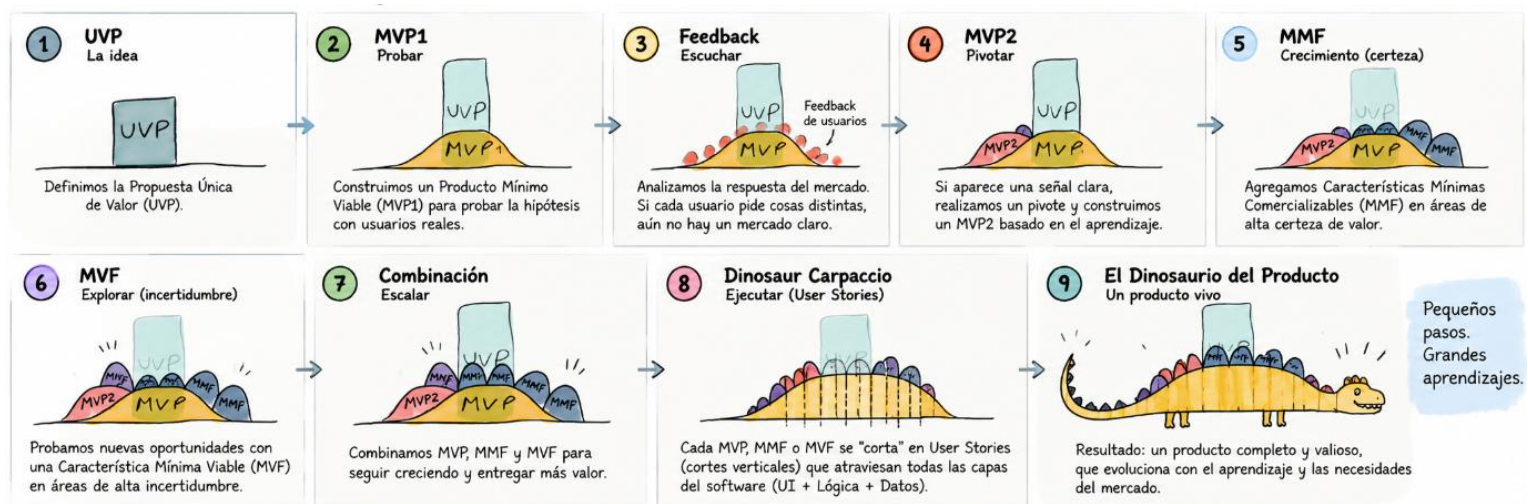
MMF — Característica Mínima Comercializable: Es la funcionalidad más pequeña que puede aportar valor comercial al usuario y a la organización. Se utiliza cuando existe alta certeza sobre lo que necesita el cliente, buscando entregar valor rápidamente y reducir el Time to Market. Puede formar parte de un lanzamiento mayor junto con otras funcionalidades. MMF = Funcionalidad individual con valor comercial.

MMP — Producto Mínimo Comercializable: Es la primera versión estable, vendible y adoptable del producto. A diferencia del MVP, ya no busca principalmente validar la idea, sino ofrecer un producto que pueda ser utilizado y adoptado por clientes reales. Está compuesto por el conjunto mínimo de MMFs necesarias y representa el primer release comercial. MMP = Primera versión para vender y adoptar.

MMR — Lanzamiento Mínimo Comercializable: Es una versión o paquete de entrega que agrupa varias MMFs y/o MVPs para incorporar nuevo valor al producto. Después del primer lanzamiento comercial (MMP = MMR 1), pueden aparecer sucesivamente MMR 2, MMR 3, etc., haciendo evolucionar el producto. MMR = Nueva versión con valor para el mercado.

Metáfora del Dinosaurio Lean/Agile

- Paso 1 – UVP: definir la idea de valor: Todo comienza con una Propuesta Única de Valor (UVP): una hipótesis sobre qué hará que el producto sea útil, diferente y valioso para los usuarios.
- Paso 2 – MVP1: probar la hipótesis: Se construye un Producto Mínimo Viable (MVP1) con lo mínimo necesario para que pueda ser utilizado por usuarios reales. Su objetivo principal es aprender y validar si existe interés en el producto, no ganar dinero inmediatamente.
- Paso 3 – Analizar la respuesta de los usuarios: Se recopila feedback. Si cada usuario pide algo completamente diferente, significa que todavía no se encontró una necesidad clara de mercado. Si muchos usuarios coinciden en que necesitan la misma funcionalidad o cambio, aparece una señal clara sobre cómo mejorar el producto.
- Paso 4 – MVP2: pivotar: Con lo aprendido se realiza un pivote, es decir, se modifica la hipótesis inicial y se construye un MVP2. Si este nuevo MVP obtiene una buena respuesta, se considera que el producto encontró una base sobre la cual continuar creciendo.
- Paso 5 – Hacer crecer el producto: A partir de ahí se agregan nuevas funcionalidades. Si existe alta certeza de que una funcionalidad aportará valor, se desarrolla una MMF (Minimum Marketable Feature), que busca entregar valor comercial rápidamente.
- Paso 6 – Experimentar cuando hay incertidumbre: Si una funcionalidad parece prometedora pero existe alta incertidumbre, se crea una MVF (Minimum Viable Feature). Es como un mini-MVP de una funcionalidad: permite probarla con usuarios reales antes de invertir demasiado. Si funciona, pueden surgir nuevas MMF; si no funciona, se descarta o modifica.
- Paso 7 – Formar el “dinosaurio”: Los MVP forman el cuerpo del dinosaurio y las MMF/MVF forman las crestas que se van agregando. Así, el dinosaurio representa un producto que crece progresivamente mediante aprendizaje y entrega de valor.
- Paso 8 – Dinosaur Carpaccio: cortar en User Stories: Aunque un MVP, MMF o MVF sea mínimo para el negocio, puede seguir siendo demasiado grande para programarlo directamente. Por eso se lo “corta en fetas”, llamadas User Stories. Cada User Story representa un pequeño corte vertical que atraviesa interfaz, lógica y persistencia y que puede desarrollarse, integrarse y probarse dentro de un Sprint.



Gestión de Configuración de Software (SCM / GCS): La Gestión de Configuración de Software (SCM / GCS) —Software Configuration Management— es una disciplina de soporte transversal a todo el proyecto y presente durante el ciclo de vida del software. Su propósito fundamental es establecer y mantener la integridad del producto, controlar su evolución y asegurar la trazabilidad de los cambios.

¿Por qué es necesaria la SCM? El software es un “blanco móvil”, ya que está sometido constantemente a cambios producidos por nuevos requerimientos, corrección de defectos, deuda técnica o evolución del negocio. Sin una gestión adecuada pueden producirse problemas como pérdida de componentes, sobreescritura de código, falta de sincronización entre código fuente y ejecutables o imposibilidad de reconstruir versiones anteriores. Texto pegado Por este motivo, SCM permite saber qué cambió, quién realizó el cambio, qué versión es la actual, qué elementos forman una determinada versión del producto y cómo recuperar estados anteriores.

Ítem de Configuración de Software (ICS / SCI): cualquier artefacto o elemento de información generado durante el ciclo de vida que forma parte del producto, proyecto o iteración, que puede sufrir modificaciones o necesita ser compartido y controlado. Para ser considerado un ICS, el artefacto debe estar digitalizado y almacenado en un repositorio. Por ejemplo, un diseño realizado únicamente en papel es un artefacto del proyecto, pero no se convierte en ICS hasta que se digitaliza. Texto pegado Los ICS se clasifican en tres grupos:

- **Ítems de Producto:** forman directamente parte de la solución. Algunos ejemplos son la ERS, arquitectura, casos de uso, código fuente, librerías, scripts de base de datos, manuales de usuario y casos de prueba.
- **Ítems de Proyecto:** corresponden a documentación administrativa general, como el Plan de Proyecto, cronograma, Plan de Calidad (QA) y Plan de SCM.
- **Ítems de Iteración o Sprint:** son artefactos propios de una etapa o iteración determinada, como el Plan de Sprint, Sprint Backlog, minutas, reportes de defectos y notas de release.

Cada ICS debe identificarse de manera unívoca mediante atributos como nombre único, regla de nombrado, versión, tipo y ubicación física dentro del repositorio.

Versión: representa el estado particular de un Ítem de Configuración en un momento determinado. El control de versiones permite conservar y seguir la evolución de cada ICS a medida que se realizan modificaciones. Por ejemplo, un archivo puede evolucionar de una versión inicial a versiones posteriores sin perder la posibilidad de conocer o recuperar sus estados anteriores.

Configuración de Software: puede entenderse como una fotografía del producto en un momento determinado. Está formada por un conjunto de Ítems de Configuración, cada uno con una versión específica. Por lo tanto, mientras la versión describe el estado de un ICS individual, la configuración describe el estado conjunto de varios ICS que conforman el producto.

Repositorio de Configuración: El repositorio es el contenedor estructurado donde se almacenan los Ítems de Configuración junto con su historial, versiones, atributos y relaciones. Además de almacenar archivos, permite realizar análisis de impacto de cambios, aplicar políticas de seguridad y acceso, realizar backups y reconstruir versiones anteriores.

Existen dos modelos principales. Los repositorios centralizados, como SVN o CVS, poseen un único servidor central donde se encuentra el repositorio. Ofrecen mayor control centralizado, pero presentan un punto único de falla. Los repositorios distribuidos, como Git, permiten que cada desarrollador tenga una copia completa del repositorio, facilitando el trabajo local y reduciendo la dependencia de un servidor central.

La interacción con el repositorio se realiza mediante operaciones como Check-Out / Update, para obtener los archivos desde el repositorio al espacio de trabajo, y Check-In / Commit, para incorporar nuevamente los cambios realizados.

Variantes: es una versión de un ICS o de una configuración que evoluciona en paralelo para satisfacer requerimientos diferentes. Por ejemplo, pueden existir variantes de un producto destinadas a distintos sistemas operativos o clientes.

Branches (ramas): son bifurcaciones de la línea principal de desarrollo que permiten trabajar de manera aislada sin desestabilizar la rama principal (main/trunk). Pueden utilizarse para desarrollar nuevas funcionalidades, realizar experimentos o corregir errores. Cuando el trabajo de una rama está terminado, puede integrarse nuevamente mediante un Merge (fusión) o descartarse si finalmente no se utilizará.

Línea Base (Baseline): conjunto de Ítems de Configuración con versiones específicas que fueron revisados y aprobados formalmente. Representa un estado estable, maduro y confiable del producto, que se utiliza como referencia para continuar el desarrollo. Las líneas base se identifican de manera unívoca dentro del repositorio mediante tags o etiquetas.

Permiten congelar y reproducir el estado del producto en un momento determinado, realizar rollback hacia versiones estables anteriores y reconstruir versiones específicas ante fallas o tareas de mantenimiento. Además, permiten determinar con precisión qué componentes forman un Release y proporcionan el estado congelado sobre el cual se realizan las Auditorías de Configuración PCA y FCA. Una vez establecida una línea base, sus elementos no pueden modificarse libremente: cualquier modificación debe seguir un procedimiento formal de Control de Cambios, autorizado por el CCB.

Existen dos tipos principales: las líneas base de especificación o documentación, correspondientes a requerimientos, arquitectura o diseño; y las líneas base operacionales, que contienen código fuente y ejecutables que superaron las pruebas de calidad y suelen estar asociadas con los releases del producto. La primera línea base operacional suele coincidir con el primer Release comercial.

Actividades fundamentales de SCM: SCM se organiza en cuatro actividades principales: Identificación de Ítems, Control de Cambios, Informes de Estado y Auditorías de Configuración.

1-Identificación de Ítems de Configuración: Es la primera actividad y consiste en determinar qué artefactos digitalizados estarán formalmente bajo control de configuración. No consiste únicamente en elegir archivos. También implica establecer una identificación unívoca, asignando nombres y versiones; definir reglas de nombrado; establecer el esquema de versionado; diseñar la estructura de carpetas del repositorio; determinar la ubicación física de cada ICS y seleccionar qué ítems maduros y aprobados formarán parte de las diferentes líneas base.

2-Control de Cambios: El Control de Cambios es el procedimiento formal utilizado para gestionar modificaciones sobre Ítems de Configuración que ya pertenecen a una Línea Base. Antes de formar parte de una línea base, un desarrollador puede modificar un ítem dentro de su espacio de trabajo siguiendo las reglas normales del equipo. Una vez incorporado a una línea base, cualquier modificación debe ser evaluada y controlada.

En el enfoque tradicional existe un Comité de Control de Cambios (CCB) integrado por representantes de distintas áreas, como líderes, desarrollo, testing, QA y clientes o Product Owner. Su función es analizar el impacto de los cambios y decidir si deben realizarse.

Flujo del Control de Cambios: El proceso comienza con una Solicitud o Propuesta de Cambio (Change Request) sobre un ICS. Luego se realiza un Análisis de Impacto para determinar cómo afectará el cambio a otros ítems, al costo y al cronograma. El CCB evalúa la solicitud y decide aprobarla o rechazarla. Si se aprueba, se genera una Orden de Cambio (OCI) con las instrucciones y restricciones correspondientes. Posteriormente, el desarrollador obtiene el ítem mediante check-out, realiza la modificación y el resultado es revisado o auditado antes de volver al repositorio mediante check-in. Finalmente, el CCB puede autorizar la actualización de la Línea Base y se notifica el cambio a los interesados.

3-Informes de Estado (Status Accounting): Los Informes de Estado registran y comunican continuamente la evolución de los Ítems de Configuración y de las solicitudes de cambio. Permiten responder preguntas como qué cambios se realizaron, quién los aprobó, qué versión implementa determinado cambio, cuáles son las diferencias entre dos versiones y cuál es la estructura actual del repositorio. También permiten generar reportes de inventario de los ICS. Texto pegado

4-Auditorías de Configuración: Las Auditorías de Configuración son evaluaciones independientes realizadas sobre una línea base congelada para comprobar que se cumple lo establecido en las especificaciones y en el Plan de SCM. Existen dos tipos principales y tienen un orden lógico: primero se realiza la Auditoría Física (PCA) y posteriormente la Auditoría Funcional (FCA).

- Auditoría Física de Configuración (PCA): La PCA —Physical Configuration Audit— realiza verificación. Comprueba que aquello que se declara en la documentación realmente exista físicamente dentro del repositorio. Revisa, por ejemplo, que los ICS se encuentren en la ubicación correcta, utilicen las reglas de nombrado establecidas, posean las versiones correspondientes y estén organizados según la estructura definida en el Plan de SCM. Si la auditoría física no es aprobada, no se continúa con la auditoría funcional.
- Auditoría Funcional de Configuración (FCA): La FCA —Functional Configuration Audit— realiza validación. Comprueba la trazabilidad entre los requerimientos definidos en la ERS y el producto efectivamente construido. Su objetivo es verificar que la funcionalidad y el desempeño real sean consistentes con lo especificado, comprobando que el código implemente todos y únicamente los requerimientos aprobados.

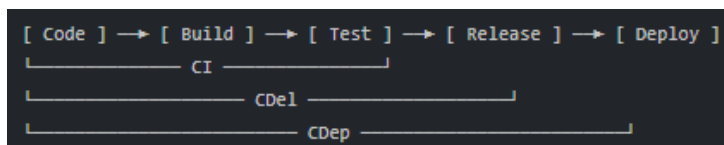
SCM en Ambientes Ágiles (Agile SCM): En los ambientes ágiles, SCM cambia su enfoque. Deja de funcionar principalmente como un mecanismo burocrático de control y pasa a convertirse en una herramienta de soporte transparente, automatizada y sin fricción para el equipo. Texto pegado

La SCM ágil busca servir al desarrollador y facilitar el cambio. En lugar de intentar evitar modificaciones mediante procedimientos pesados, permite incorporarlas de manera rápida y segura. En equipos autoorganizados no se utilizan CCB externos rígidos ni auditorías tradicionales como en el enfoque clásico. El Product Owner y el equipo gestionan los cambios mediante el backlog, mientras que muchas actividades de configuración se automatizan y se integran directamente dentro del trabajo cotidiano de cada Sprint.

También existe una responsabilidad colectiva: la gestión de configuración deja de estar aislada en una persona o sector y pasa a formar parte de las prácticas diarias del equipo.

Automatización: Continuous Integration, Continuous Delivery y Continuous Deployment

En ambientes ágiles, gran parte de SCM puede automatizarse mediante un pipeline que acompaña al software desde el código hasta su puesta en producción:



La diferencia entre Continuous Integration, Continuous Delivery y Continuous Deployment está principalmente en hasta dónde llega la automatización.

Continuous Integration (CI) — Integración Continua: La Integración Continua consiste en que los desarrolladores integren sus cambios en un repositorio compartido varias veces al día. Cada commit o push puede activar automáticamente el build y un conjunto de pruebas automatizadas, como pruebas unitarias y de integración. El objetivo es detectar rápidamente errores, regresiones y conflictos de integración, evitando acumular grandes cantidades de código sin integrar.

CI → integrar + compilar + probar continuamente.

Continuous Delivery (CDel) — Entrega Continua: La Entrega Continua extiende la Integración Continua. Después de integrar, compilar y probar el software, este se empaqueta y queda permanentemente preparado para ser desplegado en producción. La característica fundamental es que la puesta en producción todavía requiere una decisión humana. Por ejemplo, el Product Owner o una persona autorizada puede decidir cuándo liberar la nueva versión.

CDel → el software está siempre listo para producción, pero una persona decide cuándo desplegarlo.

Continuous Deployment (CDep) — Despliegue Continuo: El Despliegue Continuo lleva la automatización un paso más allá. Todo cambio que supera correctamente el pipeline de compilación y pruebas se despliega automáticamente en producción, sin requerir una aprobación humana para cada liberación.

CDep → si supera automáticamente todas las verificaciones, pasa automáticamente a producción.

Estrategias de Deployment: Cuando se utiliza despliegue continuo, pueden aplicarse estrategias que permiten liberar nuevas versiones reduciendo el riesgo y evitando interrupciones del servicio.

- **Canary Deployment:** En Canary Deployment, la nueva versión se libera primero a un grupo reducido de usuarios. Se observa su comportamiento y estabilidad y, si funciona correctamente, el despliegue se amplía progresivamente hasta alcanzar al resto de los usuarios. La idea es evitar exponer inmediatamente a toda la base de usuarios a una versión potencialmente problemática.
- **Blue/Green Deployment:** En Blue/Green Deployment se mantienen dos entornos idénticos. El entorno Blue contiene la versión actualmente utilizada por los usuarios, mientras que Green contiene la nueva versión. La nueva versión se despliega y verifica primero en Green. Cuando se comprueba que funciona correctamente, se cambia el tráfico desde Blue hacia Green. Esto permite realizar la transición rápidamente y facilita volver al entorno anterior si aparece un problema.